

## TECHNICAL REFERENCE

# TrustLance

## System Documentation — Product, Architecture & Code

|                         |                                                                                                 |
|-------------------------|-------------------------------------------------------------------------------------------------|
| Product                 | TrustLance — trust-first freelance marketplace                                                  |
| Repository              | <a href="https://github.com/TheParthi/TRTLance">github.com/TheParthi/TRTLance</a> (branch main) |
| Codebase snapshot       | commit 5d9c2268 · 5 October 2026                                                                |
| Team                    | Parthiban Gunasekaran (TheParthi) · Ponmadhan (PonmadhanD)                                      |
| Document version        | 1.0 — baseline for further development                                                          |
| Status of this document | Describes the code as it exists, including what is unfinished                                   |

---

Every statement in this document was checked against the source code at the snapshot above. Where the README or design notes describe something the code does not do, the document says so. Source files for this PDF live in `docs/technical-reference/` and can be edited and rebuilt with `build.mjs`.

# Contents

---

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| <b>1</b> | <b>Executive summary</b>                            | <b>5</b>  |
| 1.1      | Where the product stands                            | 5         |
| 1.2      | The ten most important findings                     | 6         |
| 1.3      | How to use this document                            | 6         |
| <b>2</b> | <b>Product context</b>                              | <b>7</b>  |
| 2.1      | The problem TrustLance addresses                    | 7         |
| 2.2      | Users and roles                                     | 7         |
| 2.3      | Value proposition — the four trust-first principles | 7         |
| 2.4      | Features built beyond the README                    | 8         |
| 2.5      | Positioning                                         | 8         |
| 2.6      | Scope as documented                                 | 8         |
| <b>3</b> | <b>System architecture</b>                          | <b>9</b>  |
| 3.1      | Overview                                            | 9         |
| 3.2      | Technology stack                                    | 9         |
| 3.3      | Repository structure                                | 10        |
| 3.4      | How a request flows                                 | 10        |
| 3.5      | Value stores                                        | 11        |
| <b>4</b> | <b>Authentication, sessions and roles</b>           | <b>12</b> |
| 4.1      | Sign-in methods                                     | 12        |
| 4.2      | Google sign-in flow                                 | 12        |
| 4.3      | Email sign-up flow and its failure mode             | 12        |
| 4.4      | Sessions                                            | 13        |
| 4.5      | Route protection                                    | 13        |
| 4.6      | Authorisation model                                 | 13        |
| 4.7      | Wallet linking                                      | 14        |
| <b>5</b> | <b>Database and data model</b>                      | <b>15</b> |
| 5.1      | Source files and the order they run in              | 15        |
| 5.2      | Entity-relationship overview                        | 16        |
| 5.3      | Table catalogue                                     | 16        |
| 5.4      | Row-level security                                  | 18        |
| 5.5      | Functions, triggers, storage and realtime           | 19        |
| 5.6      | State machines                                      | 19        |
| 5.7      | Data-model issues to fix first                      | 19        |
| <b>6</b> | <b>API reference</b>                                | <b>21</b> |
| 6.1      | Conventions                                         | 21        |
| 6.2      | Endpoint summary                                    | 21        |
| 6.3      | Jobs and bids                                       | 22        |
| 6.4      | Contracts and milestones                            | 23        |
| 6.5      | Disputes and arbitration                            | 23        |
| 6.6      | Wallet                                              | 24        |

|           |                                                              |          |
|-----------|--------------------------------------------------------------|----------|
| 6.7       | Reviews, feedback and profile .....                          | 3        |
| 6.8       | Business rules: where they are enforced .....                | 3        |
| <b>7</b>  | <b>Blockchain, tokens and payments .....</b>                 | <b>3</b> |
| 7.1       | Smart contracts .....                                        | 3        |
| 7.2       | Networks and deployments .....                               | 3        |
| 7.3       | The platform token .....                                     | 3        |
| 7.4       | End-to-end money flow .....                                  | 3        |
| 7.5       | Wallet integration .....                                     | 3        |
| 7.6       | Payments that are simulated .....                            | 3        |
| 7.7       | Security of funds — summary .....                            | 3        |
| <b>8</b>  | <b>AI features .....</b>                                     | <b>3</b> |
| 8.1       | Inventory .....                                              | 3        |
| 8.2       | A. Project Risk Analysis .....                               | 3        |
| 8.3       | B. AI project recommendations (unused) .....                 | 3        |
| 8.4       | C. LangChain dispute agent (unused) .....                    | 3        |
| 8.5       | D–H. Simulated AI .....                                      | 3        |
| 8.6       | Recommended AI roadmap .....                                 | 3        |
| <b>9</b>  | <b>Frontend .....</b>                                        | <b>3</b> |
| 9.1       | Application shell .....                                      | 3        |
| 9.2       | Navigation .....                                             | 3        |
| 9.3       | Page inventory .....                                         | 3        |
| 9.4       | Design system .....                                          | 3        |
| 9.5       | State and data .....                                         | 3        |
| 9.6       | Responsiveness and accessibility .....                       | 3        |
| 9.7       | Frontend defects that block a build .....                    | 3        |
| <b>10</b> | <b>End-to-end user journeys .....</b>                        | <b>3</b> |
| 10.1      | Journey 1 — Joining TrustLance .....                         | 3        |
| 10.2      | Journey 2 — From project to payment (the core flow) .....    | 3        |
| 10.3      | Journey 3 — Disputes .....                                   | 3        |
| 10.4      | Journey 4 — Wallet and tokens .....                          | 3        |
| 10.5      | Journey 5 — Messaging .....                                  | 3        |
| 10.6      | Journey 6 — Reputation .....                                 | 3        |
| <b>11</b> | <b>Configuration, environments and running locally .....</b> | <b>3</b> |
| 11.1      | Environment variables .....                                  | 3        |
| 11.2      | Scripts and ports .....                                      | 3        |
| 11.3      | Build configuration .....                                    | 3        |
| 11.4      | Setting up a working local environment .....                 | 3        |
| 11.5      | Dependency notes .....                                       | 3        |
| <b>12</b> | <b>Project history .....</b>                                 | <b>3</b> |
| 12.1      | Timeline .....                                               | 3        |
| 12.2      | Contributors .....                                           | 3        |
| 12.3      | Pivots .....                                                 | 3        |
| <b>13</b> | <b>Implementation status .....</b>                           | <b>3</b> |

|           |                                                           |          |
|-----------|-----------------------------------------------------------|----------|
| 13.1      | Accounts and identity .....                               | 4        |
| 13.2      | Marketplace .....                                         | 4        |
| 13.3      | Money .....                                               | 4        |
| 13.4      | Disputes .....                                            | 4        |
| 13.5      | Platform .....                                            | 4        |
| <b>14</b> | <b>Security review .....</b>                              | <b>4</b> |
| 14.1      | High .....                                                | 4        |
| 14.2      | Medium .....                                              | 4        |
| 14.3      | Low .....                                                 | 4        |
| 14.4      | Priority order for remediation .....                      | 4        |
| <b>15</b> | <b>Known issues .....</b>                                 | <b>4</b> |
| 15.1      | Build blockers .....                                      | 4        |
| 15.2      | Functional bugs .....                                     | 4        |
| 15.3      | Documentation that contradicts the code .....             | 4        |
| 15.4      | Housekeeping .....                                        | 4        |
| <b>16</b> | <b>Roadmap: from prototype to product .....</b>           | <b>4</b> |
| 16.1      | Phase 0 — Stabilise the codebase (about 1 week) .....     | 4        |
| 16.2      | Phase 1 — Security baseline (1–2 weeks) .....             | 4        |
| 16.3      | Phase 2 — One trustworthy money layer (3–4 weeks) .....   | 4        |
| 16.4      | Phase 3 — Disputes that work end to end (2–3 weeks) ..... | 4        |
| 16.5      | Phase 4 — Communication (1–2 weeks) .....                 | 4        |
| 16.6      | Phase 5 — Product polish (2–3 weeks) .....                | 4        |
| 16.7      | Phase 6 — Production readiness .....                      | 4        |
| 16.8      | Suggested first sprint .....                              | 4        |
| <b>17</b> | <b>Appendix .....</b>                                     | <b>4</b> |
| 17.1      | Glossary .....                                            | 4        |
| 17.2      | Key files .....                                           | 4        |
| 17.3      | Frontend → API map .....                                  | 4        |
| 17.4      | Contract addresses .....                                  | 4        |
| 17.5      | About this document .....                                 | 4        |

# 1 Executive summary

TrustLance is a freelance marketplace built around one idea: neither side of a freelance job should have to *trust* the other side or the platform. Clients lock the budget before work starts, freelancers are paid milestone by milestone when work is approved, identities are tied to a crypto wallet, and disagreements go to an arbitrator through a recorded process.

The codebase is a **Next.js 15 web application** with a **Supabase (PostgreSQL)** backend, **Solidity smart contracts** on **Shardeum** and Polygon Amoy, and **Gemini-based AI** for project risk analysis. It was built in two intense sprints — 24–27 January 2026 and 14–15 February 2026 — by Ponmadhan (PonmadhanD) and Parthiban Gunasekaran (TheParthi), and won the GDG KSR Web3 Hackathon run with Shardeum.

|                                                               |                                             |                                            |                                                |
|---------------------------------------------------------------|---------------------------------------------|--------------------------------------------|------------------------------------------------|
| <b>47</b><br>pages (Next.js App Router)                       | <b>37</b><br>API handlers in 32 route files | <b>31,350</b><br>lines of TypeScript / TSX | <b>3</b><br>Solidity contracts (426 lines)     |
| <b>27 + 6</b><br>database tables defined + used-but-undefined | <b>13</b><br>SQL files (1,316 lines)        | <b>35</b><br>commits, 2 authors            | <b>1</b><br>live AI feature (Gemini 2.0 Flash) |

## 1.1 Where the product stands

TrustLance is a **working hackathon prototype**, not a production system. A narrow path through the product works end to end against real services; most of the surrounding screens are designed but not yet connected.

### What works today (the "spine"):

- Sign-in with Google or email/password (Supabase Auth), automatic profile creation.
- A client posts a project through a 9-step wizard, signs an agreement on screen and **locks the budget on-chain in SHM** on Shardeum (chain 8118).
- A freelancer opens the job, gets an **AI risk report** (Gemini 2.0 Flash) and submits a bid with milestones.
- The client accepts a bid, which creates a **contract** and debits the client's platform-token balance.
- Inside the contract: both parties sign, the freelancer submits proof of work, the client requests revisions or **releases a milestone**, the freelancer is credited, and both leave reviews.
- Platform token wallet (buy, redeem, history), notifications with realtime updates, public profiles, and an arbitrator dashboard with time-slot booking and a dispute room.

### What is not finished:

- **The on-chain escrow and the platform ledger are disconnected.** SHM locked when a project is posted can only be withdrawn by the contract owner; freelancers are paid in off-chain database tokens instead. The full milestone escrow contract ( [ProjectEscrow](#) ) exists and is deployed on Polygon Amoy but the app never calls it.
- **Disputes are mostly a designed UI on mock data.** No screen creates a real dispute; the arbitrator room works only for disputes inserted by other means, and resolving one moves no money.
- **Encrypted XMTP messaging, PayPal escrow and real token purchases do not exist** — they are described in the README but not implemented.
- **The repository cannot be rebuilt as-is:** two pages contain unresolved merge-conflict markers, three files import a module that does not exist, and the database SQL in the repo is missing six tables and about a

dozen columns that the code uses.

## 1.2 The ten most important findings

| #  | Finding                                                                                                        | Impact                            | Chapter |
|----|----------------------------------------------------------------------------------------------------------------|-----------------------------------|---------|
| 1  | Escrowed SHM has no release or refund path; only the deployer can withdraw it                                  | Client funds stranded / custodial | 7       |
| 2  | Milestone release can be repeated and amounts are editable by either party                                     | Unlimited balance inflation       | 7, 14   |
| 3  | Buying tokens needs no payment; accepting a bid auto-tops the client up to 1,000 tokens                        | Free value creation               | 7, 14   |
| 4  | Every user can read every other user's row (email, phone, balance) and edit their own balance, rating and role | Privacy breach, self-promotion    | 5, 14   |
| 5  | Six tables and ~12 columns used by the code are not in the repo's SQL                                          | Database cannot be recreated      | 5       |
| 6  | Merge-conflict markers and a missing module break <code>next build</code>                                      | Cannot deploy                     | 15      |
| 7  | Wallet ownership is never verified (the signature is discarded)                                                | Anyone can claim any address      | 4       |
| 8  | AI risk analysis stores a fabricated "LOW risk" report whenever the model call fails                           | Misleading users                  | 8       |
| 9  | Dispute screens, admin console and "AI verdicts" are hard-coded mock data                                      | Core trust feature not functional | 9, 10   |
| 10 | README claims (Sepolia, TRT token, XMTP, PayPal, Google-only login) do not match the code                      | Misleading documentation          | 15      |

## 1.3 How to use this document

- **Chapters 2–3** explain what TrustLance is for and how it is put together.
- **Chapters 4–9** are the technical reference: authentication, database, API, blockchain, AI and frontend, each describing exactly what the code does today.
- **Chapter 10** walks through every end-to-end user journey step by step.
- **Chapters 11–12** cover configuration, running the project and its history.
- **Chapters 13–15** are the honest status: a feature matrix, the security review and known issues.
- **Chapter 16** is a phased roadmap for turning the prototype into a product.

### Conventions

File references are written as `path:line` relative to the repository root. Status badges: Working real logic against real services · Partial works with gaps · Mock hard-coded or simulated · Missing described but not built or broken.

## 2 Product context

### 2.1 The problem TrustLance addresses

Freelance work runs on trust that neither party can verify. The README ( [README.md:8-17](#) ) names five recurring failures of traditional freelance platforms:

- **Payment trust** — freelancers start work without knowing the money exists.
- **Delayed or denied payments** — clients hold back payment after delivery.
- **Opaque dispute handling** — the platform decides behind closed doors.
- **Fake or unverifiable identities** — anyone can create accounts.
- **Platform-controlled messaging** — the platform can read and moderate everything.

"Both clients and freelancers operate with uncertainty." TrustLance's answer is a **workflow-first** marketplace in which each of these risks is removed by design rather than by reputation.

### 2.2 Users and roles

| Role              | What they do                                                                                                                           | Where it lives in the code                                                                                                      |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <b>Client</b>     | Posts projects, defines the budget, hires a freelancer, signs the agreement, approves milestones and releases payment, raises disputes | Per record: <code>projects.client_id</code> , <code>contracts.client_id</code>                                                  |
| <b>Freelancer</b> | Finds projects, checks the AI risk report, bids with milestones, signs, submits proof of work, gets paid                               | Per record: <code>proposals.freelancer_id</code> , <code>contracts.freelancer_id</code>                                         |
| <b>Arbitrator</b> | Any user with $\geq 3,000$ tokens can book time slots, go online, be assigned disputes and decide them (release or refund)             | <code>/resolution-gigs</code> , <code>users.tokens</code> , <code>users.arbitrator_status</code> , <code>arbitrator_gigs</code> |
| <b>Admin</b>      | Intended to monitor disputes and override decisions                                                                                    | Only a mock page ( <code>/admin/disputes</code> ); no admin role exists                                                         |

Roles are **contextual, not global**: every account is created as `freelancer`, and a person is a "client" on the projects they posted and a "freelancer" on the contracts they were hired for. The global `users.role` column is displayed but never used for authorisation.

### 2.3 Value proposition — the four trust-first principles

The README promises four mechanisms. The table shows how far each one is actually built.

| Principle                 | Promise                                                                 | What the code does                                                                                                                                                                                                 | Status         |
|---------------------------|-------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| <b>Escrow before work</b> | Funds are locked before work begins and released per approved milestone | Budget is locked on-chain at posting (Shardeum <code>Escrow.LockFunds</code> ), but that contract cannot release or refund. Hiring separately debits off-chain tokens; milestone release credits off-chain tokens. | <b>Partial</b> |
| <b>Verified identity</b>  | Google sign-in plus a one-time, irreversible wallet link                | Google OAuth works (email/password also exists). Wallet linking is enforced only in browser code, ownership is never proven, and the database lets users change the address.                                       | <b>Partial</b> |
| <b>Private messaging</b>  | End-to-end encrypted wallet-to-wallet chat via XMTP, only after hiring  | No XMTP at all. Contract chat is plain text in Supabase ( <code>contract_messages</code> ); the inbox and floating chat are hard-coded.                                                                            | <b>Missing</b> |

| Principle                 | Promise                                                            | What the code does                                                                                                                                                                                | Status      |
|---------------------------|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <b>Auditable disputes</b> | Clear, recorded dispute process with AI analysis and expert review | A <code>disputes</code> table, arbitrator assignment, a dispute chat and a resolve endpoint exist, but no UI creates disputes, resolution moves no money and the AI/expert screens are mock data. | Mostly mock |

## 2.4 Features built beyond the README

The code contains several product areas the README does not mention — and in two cases explicitly rules out:

- **Platform token economy** — 1 token = ₹10; buy, redeem, balance and transaction history (`/profile/wallet`, `/api/wallet/*`).
- **AI project risk analysis** — a Gemini report shown to freelancers before they bid, downloadable as PDF.
- **Digital signatures and agreement PDFs** — drawn signatures on the project and contract, PDF export with jsPDF.
- **Reviews, ratings and "trust credits"** — two-way feedback, portfolio items created automatically from client reviews, +30 credits per completed project. (The README lists ratings as out of scope.)
- **Rich public profiles** — `/u/[username]` with education, certifications, portfolio, reviews and earnings.
- **Arbitrator marketplace ("Resolution Gigs")** — a token-gated role with bookable time slots.

## 2.5 Positioning

No document in the repo names competitors, but the design positions TrustLance against custodial marketplaces such as Upwork and Fiverr. Those platforms hold client money in their own accounts, run in-house mediation and rely on reputation scores. TrustLance proposes a **smart-contract escrow**, a **wallet-bound identity** and a **transparent dispute trail** instead, with the backend README pitching on-chain fees of about \$0.10 per project against PayPal's 2.9%.

### Brand consistency

The shipped UI copy still borrows heavily from Freelancer.com: the homepage and page metadata say "World's largest freelance marketplace" and "Save up to 90%", the header search box reads "Search Freelancer.com" (`src/components/layout/authenticated-header.tsx:342`), and the footer claims "60,000,000+" users. These should be replaced with TrustLance's own positioning before any public launch.

## 2.6 Scope as documented

The README draws an intentional scope boundary (`README.md:119-145`):

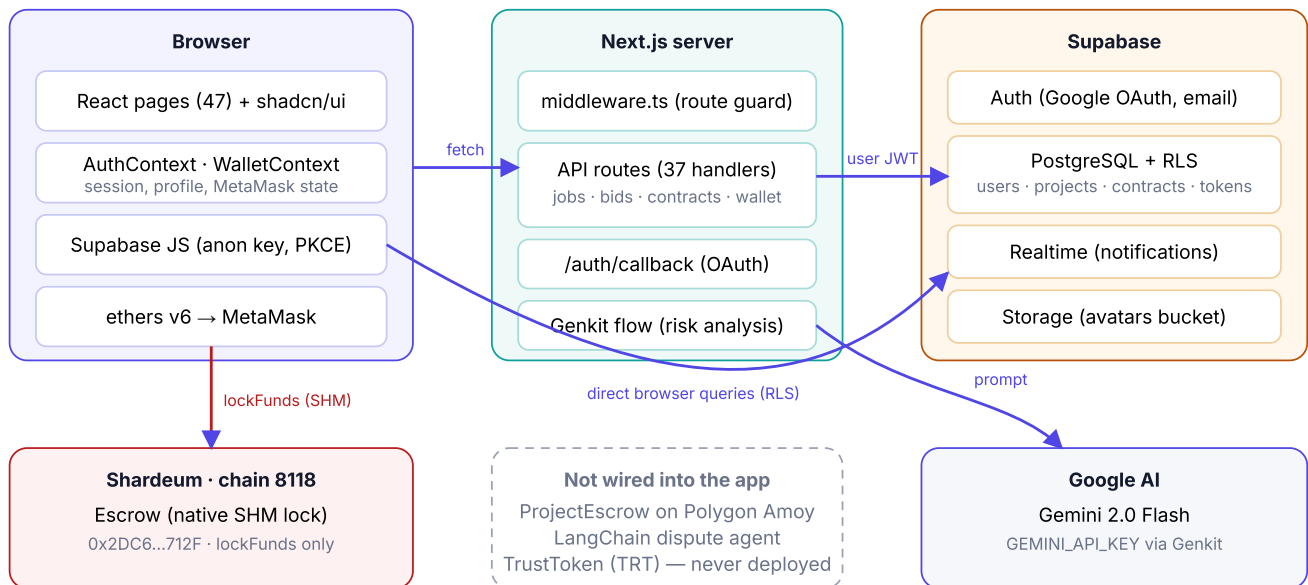
- **In scope:** job posting, proposals, milestone escrow, wallet identity, encrypted messaging, disputes, demo/testnet deployment.
- **Out of scope:** ratings and reviews, advanced AI moderation, mainnet deployment, social features.

In practice, ratings and reviews were built, the app is configured against chain 8118 which the config itself labels "Shardeum Mainnet", and encrypted messaging was not built. Chapter 15 lists every documentation gap.

## 3 System architecture

### 3.1 Overview

TrustLance is a single **Next.js 15 (App Router)** application. The same codebase serves the pages (React 19 client components) and the backend (API route handlers under `src/app/api`). Persistent state lives in **Supabase**; the browser talks to it both directly (through the public anon key, guarded by row-level security) and through the API routes. Money moves on two separate rails: native **SHM on Shardeum** through the user's MetaMask, and an **off-chain token ledger** in Supabase.



#### Two money rails that never meet

- On-chain: client's SHM → Escrow contract at project posting (cannot be released to the freelancer).
- Off-chain: `user_wallets.token_balance` debited at hiring, credited at milestone release (1 token = ₹10).
- Service-role key is used server-side only, in one route (milestone release) and in maintenance scripts.

Figure 3.1 — Runtime components and data flows as implemented

### 3.2 Technology stack

| Layer              | Technology                                                         | Version       | Notes                                                                 |
|--------------------|--------------------------------------------------------------------|---------------|-----------------------------------------------------------------------|
| Framework          | Next.js App Router                                                 | 15.5.9        | Dev server on Turbopack, port 9002                                    |
| UI runtime         | React                                                              | 19.2          | Almost every page is a client component ( <code>"use client"</code> ) |
| Styling            | Tailwind CSS + shadcn/ui (Radix)                                   | 3.4           | Indigo <code>#4F46E5</code> primary, Inter + Source Code Pro          |
| Icons, charts, PDF | lucide-react, recharts, jsPDF + autotable                          | —             | Charts use hard-coded data                                            |
| Auth & database    | Supabase ( <code>@supabase/ssr</code> , <code>supabase-js</code> ) | 0.8 / 2.91    | Auth, PostgreSQL with RLS, Realtime, Storage                          |
| Web3               | ethers v6 + MetaMask ( <code>window.ethereum</code> )              | 6.16          | No wagmi/viem/WalletConnect                                           |
| Smart contracts    | Solidity + Hardhat + OpenZeppelin 5                                | 0.8.20 / 2.28 | In <code>backend/</code> (own <code>package.json</code> )             |

| Layer              | Technology                                  | Version | Notes                                                                                |
|--------------------|---------------------------------------------|---------|--------------------------------------------------------------------------------------|
| AI (live)          | Genkit + Google AI plugin                   | 1.20    | Model <code>googleai/gemini-2.0-flash</code>                                         |
| AI (not wired)     | LangChain JS (OpenAI / Ollama)              | 0.1     | <code>backend/ai/dispute-agent.js</code>                                             |
| Hosting (template) | Firebase App Hosting, Firebase Studio (IDX) | —       | <code>apphosting.yaml</code> , <code>.idx/dev.nix</code> ; no evidence of deployment |

### 3.3 Repository structure

```

TRTLance/
├── src/
│   ├── app/
│   │   ├── api/
│   │   │   ├── arbitrator, feedback, reviews, profile,
│   │   │   │   projects, wallet, ai
│   │   │   └── auth/callback/
│   │   │       Google OAuth code exchange + profile creation
│   │   │       ...
│   │   ├── components/
│   │   │   80 components (36 shadcn/ui primitives)
│   │   ├── contexts/
│   │   │   auth-context, wallet-context, chat-context (unused)
│   │   ├── ai/
│   │   │   Genkit setup and two flows
│   │   ├── hooks/
│   │   │   use-toast, mock dispute/workspace hooks
│   │   ├── lib/
│   │   │   supabase clients, config, contracts, payments, roles
│   │   ├── scripts/
│   │   │   demo-user and diagnostic scripts (service role)
│   │   └── middleware.ts
│   │       server-side route protection
│   ├── backend/
│   │   ├── contracts/
│   │   │   Escrow.sol, ProjectEscrow.sol, TrustToken.sol
│   │   ├── scripts/
│   │   │   Hardhat deploy scripts (Amoy, Shardeum)
│   │   ├── deployments/
│   │   │   recorded contract addresses
│   │   ├── ai/dispute-agent.js
│   │   │   LangChain dispute recommender (unused)
│   │   └── supabase/
│   │       schema, RLS, migrations, seed SQL
│   ├── blockchain/contracts/
│   │   older copies of Escrow and TrustToken
│   ├── docs/
│   │   blueprint + this technical reference
│   └── README.md, SETUP_STEPS.md, DISPUTE_RESOLUTION_SYSTEM.md, GET_PRIVATE_KEY.md

```

### 3.4 How a request flows

- Page load.** `src/middleware.ts` runs on every request (including `/api/*`), refreshes the Supabase session cookie and redirects unauthenticated users away from nine protected prefixes. `ConditionalLayout` repeats the check in the browser for a longer list of prefixes and chooses the public or authenticated shell.
- Data fetching.** Pages fetch in `useEffect` on mount — there is no caching layer. About half the data comes from `/api/*` routes; the rest comes from direct browser queries to Supabase (`users`, `notifications`, `feedback`, `user_education`, `transactions`, storage).
- API routes.** Each handler creates a server Supabase client from the request cookies, calls `supabase.auth.getUser()` to verify the JWT, checks ownership where implemented, and reads or writes as the user — so **row-level security applies**. One route (milestone release) additionally uses the service-role client to write ledger rows that RLS would block.
- Blockchain.** Only one on-chain write happens anywhere: `Escrow.lockFunds` when a project is posted, sent from the user's MetaMask. No API route touches the chain.
- AI.** `POST /api/ai/project-risk` runs a Genkit flow in-process, which calls Gemini and caches the result in `ai_project_risk_reports`.

### 3.5 Value stores

The same idea of "money" is held in five places that are not reconciled with each other:

| Store                                                                                           | Unit              | Written by                                                         | Purpose                                     |
|-------------------------------------------------------------------------------------------------|-------------------|--------------------------------------------------------------------|---------------------------------------------|
| MetaMask balance on chain 8118                                                                  | SHM               | the user                                                           | Shown in headers and wallet pages           |
| <code>Escrow</code> contract balance                                                            | SHM               | <code>lockFunds</code> at project posting                          | Budget lock; withdrawable only by the owner |
| <code>user_wallets.token_balance</code> + <code>token_transactions</code>                       | tokens (₹10 each) | buy, redeem, bid accept, milestone release                         | The working platform ledger                 |
| <code>users.balance</code> + <code>transactions</code>                                          | ₹                 | DB trigger on <code>transactions</code> (fed by milestone release) | Legacy fiat ledger                          |
| <code>users.total_credits</code> + <code>credit_transactions</code> , <code>users.tokens</code> | credits / tokens  | reviews; manual                                                    | Reputation credits; arbitrator eligibility  |

Chapter 7 traces how value moves between them, and chapter 16 proposes collapsing them into a single ledger.

## 4 Authentication, sessions and roles

### 4.1 Sign-in methods

| Method                     | Implementation                                                                                                                                                                                                            | Status  |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| Google OAuth               | <code>supabase.auth.signInWithOAuth({ provider: 'google', redirectTo: origin + '/auth/callback' })</code> with the PKCE flow ( <code>src/contexts/auth-context.tsx:234-255</code> , <code>src/lib/supabase.ts:12</code> ) | Working |
| Email + password           | <code>signup</code> / <code>signInWithPassword</code> ( <code>auth-context.tsx:155-232</code> ) — contradicts the README's "Google only"                                                                                  | Partial |
| Wallet login (SIWE)        | Not implemented. MetaMask <code>personal_sign</code> is requested on connect but the signature is discarded ( <code>wallet-context.tsx:113-120</code> )                                                                   | Missing |
| Password reset, magic link | "Forgot password?" links to <code>#</code> ( <code>login-form.tsx:132</code> )                                                                                                                                            | Missing |

The login field is labelled "Email or Username", but only email works; "Remember me" and the Terms checkbox do nothing. Facebook login was removed before the first commit.

### 4.2 Google sign-in flow

1. The browser starts the PKCE flow; the code verifier is stored in a cookie by the Supabase SSR browser client.
2. Google returns to Supabase, which redirects to `GET /auth/callback?code=...`.
3. `src/app/auth/callback/route.ts` exchanges the code for a session ( `exchangeCodeForSession` ) and writes the session cookies.
4. It looks up the `users` row. If none exists it **generates a username** from the email local part (lower-cased, non-alphanumerics replaced with `_`, then `name`, `name_2`, `name_3`... until free) and inserts `{ id, email, name, username, role: 'freelancer', avatar_url, membership_tier: 'free', balance: 0, currency: 'INR' }`. Insert errors are logged and ignored.
5. It always redirects to `/dashboard`. In the browser, `onAuthStateChange('SIGNED_IN')` loads the profile into `AuthContext`.
6. A database trigger ( `handle_new_user_wallet` ) creates the user's `user_wallets` row with a balance of 0.

### 4.3 Email sign-up flow and its failure mode

`signup()` calls `supabase.auth.signup`, generates a username in the browser and **inserts the `users` row from the browser** ( `auth-context.tsx:205-216` ), relying on the RLS rule `auth.uid() = id`.

#### Profile row can be missing

If Supabase email confirmation is enabled, `signup` returns no session, so the browser insert is rejected and the account has no profile. The user can still log in later, but `login()` never creates a profile, and because the confirmation link does not go to `/auth/callback`, nothing else will either. Middleware then sees a session while `AuthContext` sees no user, which can bounce the user between `/login` and `/dashboard`. Fix: create profiles in a `SECURITY DEFINER` trigger on `auth.users`.

## 4.4 Sessions

- **Browser:** `AuthProvider` reads the session with `getSession()` on mount and subscribes to `onAuthStateChange`. `autoRefreshToken` keeps the access token fresh and rewrites the cookies.
- **Server:** every authenticated API route calls `supabase.auth.getUser()`, which validates the JWT with Supabase Auth — the correct pattern; `getSession()` is never trusted server-side.
- **Middleware** (`src/middleware.ts`) refreshes expired tokens on every request using the standard `@supabase/ssr` cookie pattern. Its matcher includes `/api/*`, so each API call costs two Auth round trips.
- **Logout:** `supabase.auth.signOut()` and a redirect to `/login`.

## 4.5 Route protection

| Layer                                 | Protects                                                                                                                                                                                                                                         | Mechanism                                                                                                               |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Server middleware                     | <code>/dashboard</code> , <code>/profile/*</code> , <code>/wallet</code> , <code>/contracts</code> , <code>/inbox</code> , <code>/settings</code> , <code>/lists</code> , <code>/tasklists</code> , <code>/post-project</code>                   | Redirect to <code>/login</code> ; logged-in users are redirected away from <code>/login</code> and <code>/signup</code> |
| Client <code>ConditionalLayout</code> | A 44-prefix list (many of them routes that don't exist)                                                                                                                                                                                          | Spinner, then <code>router.replace('/login')</code>                                                                     |
| Unprotected server-side               | <code>/admin/*</code> , <code>/disputes/*</code> , <code>/resolution-gigs/*</code> , <code>/job/*</code> , <code>/my-projects</code> , <code>/notifications</code> , <code>/feedback</code> , <code>/u/*</code> , <code>/payment-demo</code> ... | Client guard only, or none — <code>/admin/disputes</code> is reachable by anyone                                        |
| API routes                            | Each handler checks for itself                                                                                                                                                                                                                   | Several have no check at all (see chapter 6)                                                                            |

## 4.6 Authorisation model

TrustLance authorises **per record**, not per role:

- A project's owner is `projects.client_id`. Only the owner can view its bids, accept a bid or edit it.
- A contract's parties are `contracts.client_id` and `contracts.freelancer_id`. Only parties can view or sign it; only the freelancer can submit milestones; only the client can request revisions or release payment.
- A dispute's decision-maker is `disputes.arbitrator_id`.
- `src/lib/roles.ts` provides client-side helpers (`getUserRoleInJob`, `canApplyToJob`) used to choose which view a page shows.

The global `users.role` column (`client | freelancer | both`) is always `freelancer` at creation and is **never checked** by any API route.

### Arbitrator

There is no single arbitrator flag. Eligibility is `users.tokens ≥ 3000` (checked only when booking a slot), availability is `users.arbitrator_status` (any user can set themselves online), and authority over a specific dispute is `disputes.arbitrator_id`. Constants in `src/lib/arbitrator.ts` (30 tokens per project, ₹10 cancellation penalty, 1-hour booking advance, 3 specialisations) are mostly not enforced. Both `users.tokens` and `users.arbitrator_status` are missing from the repo's SQL.

### Admin

There is no admin role, claim, table or API. "Admin" power exists only as the Supabase service-role key on the server.

## 4.7 Wallet linking

`LinkWallet` ( `auth-context.tsx:264-310` ) runs entirely in the browser:

1. Lower-case the address.
2. Refuse if another user already has it ("already linked to @username" — this also reveals the other account's username).
3. Refuse if the current user already linked a different wallet ("binding is permanent").
4. `UPDATE users SET wallet_address = ...` directly from the browser.

| Promise                           | Reality                                                                                                             |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------|
| One wallet per account, permanent | Enforced only by browser code; RLS lets a user rewrite or clear <code>wallet_address</code> at any time             |
| Wallet proves identity            | Ownership is never verified — the signed message is thrown away                                                     |
| Real wallet                       | Without MetaMask, a hard-coded demo address <code>0x742d35Cc...</code> is "connected" and can be linked permanently |
| Wallet used for payments          | The linked address is never used by any money flow                                                                  |

The database's only server-side protection is `UNIQUE(wallet_address)`, which is case-sensitive.

## 5 Database and data model

### 5.1 Source files and the order they run in

All database code is plain SQL in `backend/supabase/`, applied by hand in the Supabase SQL editor. There is no `supabase/config.toml` and no migration tracking.

| File                                                        | What it creates                                                                                                                                      |
|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>reset.sql</code>                                      | Drops 14 base tables and two functions (incomplete — leaves 13 later tables behind)                                                                  |
| <code>schema.sql</code> (416 lines)                         | 14 base tables, <code>contracts</code> , the token wallet tables, indexes, triggers, a wallet backfill                                               |
| <code>rls_policies.sql</code> (264 lines)                   | Row-level security for 13 tables; <b>lines 228–264 are UTF-16 encoded</b> and contain the only definition of <code>contract_messages</code>          |
| <code>feedback_schema.sql</code>                            | Two-way feedback tables, rating trigger, helper functions                                                                                            |
| <code>apply_signatures_migration.sql</code>                 | Signature columns (no-op on a fresh schema)                                                                                                          |
| <code>seed_sample_contracts.sql</code>                      | Demo project + contract (broken: uses <code>auth.uid()</code> , which is NULL in the editor)                                                         |
| <code>migrations/20240126_add_client_signing.sql</code>     | <code>contracts.client_signature</code> , <code>client_signed_at</code>                                                                              |
| <code>migrations/20240130_create_risk_reports.sql</code>    | <code>ai_project_risk_reports</code>                                                                                                                 |
| <code>migrations/20240214_add_project_date_range.sql</code> | <code>projects.start_date</code> , <code>end_date</code> — <b>required</b> by <code>POST /api/jobs</code>                                            |
| <code>migrations/20240214_create_wallet_system.sql</code>   | Exact duplicate of the wallet section of <code>schema.sql</code> (cannot run both)                                                                   |
| <code>migrations/20240215_add_profile_fields.sql</code>     | <code>users.hourly_rate</code> , <code>company</code> , <code>phone</code>                                                                           |
| <code>migrations/20240215_create_feedback_back.sql</code>   | Generic <code>feedback</code> table                                                                                                                  |
| <code>migrations/20240215_rich_profile.sql</code>           | Profile columns and <code>user_education</code> , <code>user_certifications</code> , <code>portfolio_items</code> , <code>credit_transactions</code> |

#### The repository cannot recreate the live database

The setup guide says to run only `schema.sql` then `rls_policies.sql`, which leaves out every migration the code depends on. Worse, **six tables** used by the code exist in no SQL file at all —

`milestone_submissions`, `dispute_messages`, `arbitrator_gigs`, `arbitrators`, `profiles`, `site_reviews` — along with three RPCs (`release_escrow_to_freelancer`, `refund_escrow_to_client`, `deduct_penalty`) and about a dozen columns. Either the live Supabase project contains objects that were never committed, or those features are broken. The first step of any further work is `supabase db pull` to capture the real schema.

## 5.2 Entity-relationship overview

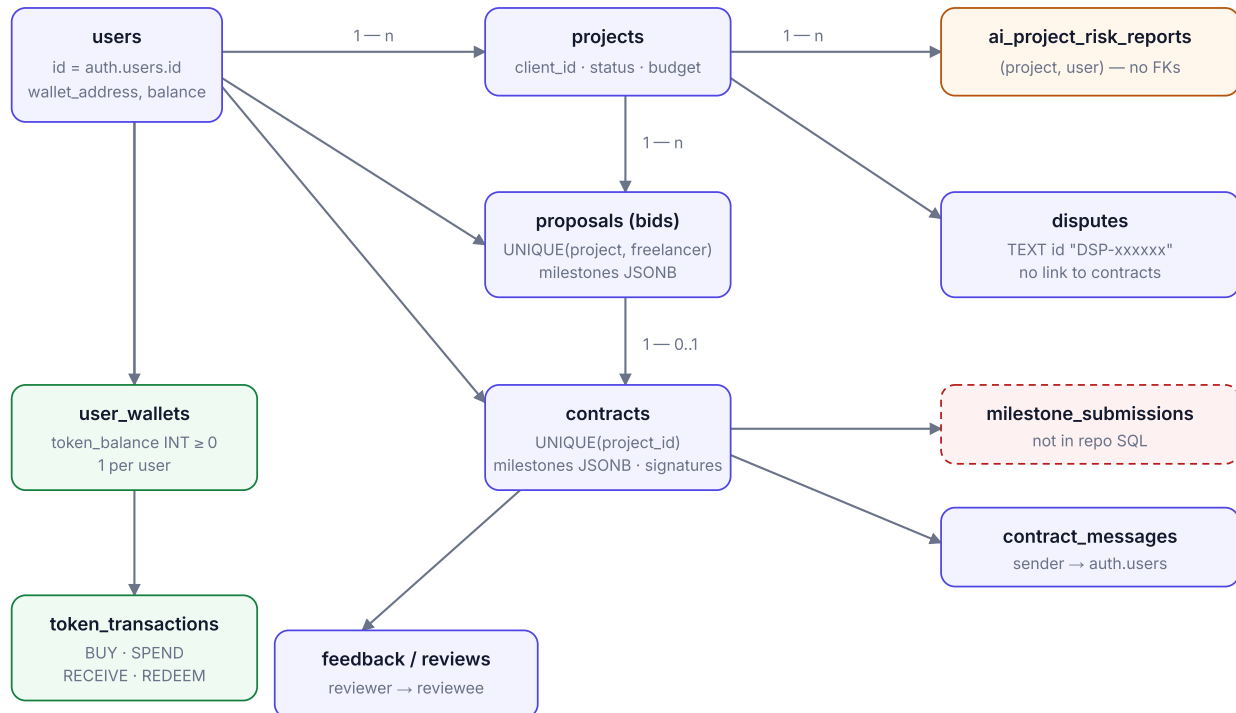


Figure 5.1 — Core entities. Dashed = used by code but not defined in the repo's SQL.

## 5.3 Table catalogue

### Marketplace core

| Table                              | Purpose                                                                  | Key columns                                                                                                                                                                                                                                                                                                                                                                                                                                                | Used? |
|------------------------------------|--------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| <code>users</code>                 | Profile mirroring<br><code>auth.users</code><br>(same id, <b>no FK</b> ) | <code>email</code> , <code>username</code> (unique), <code>role</code> , <code>wallet_address</code> (unique),<br><code>balance</code> , <code>rating</code> , <code>membership_tier</code> , profile fields ( <code>about</code> ,<br><code>skills</code> , <code>languages</code> , <code>hourly_rate</code> , ...)                                                                                                                                      | Yes   |
| <code>projects</code>              | A job post;<br>becomes the<br>project once<br>someone is hired           | <code>client_id</code> , <code>title</code> , <code>description</code> , <code>category</code> , <code>budget_type</code><br>fixed/hourly, <code>budget_min/max</code> , <code>experience_level</code> , <code>skills[]</code> ,<br><code>visibility</code> , <code>status</code> , <code>hired_freelancer_id</code> , <code>client_signature</code> ,<br><code>start_date</code> , <code>end_date</code>                                                  | Yes   |
| <code>proposals</code>             | A freelancer's bid<br>("bids" in the<br>API)                             | <code>project_id</code> , <code>freelancer_id</code> , <code>cover_letter</code> , <code>proposed_budget</code> ,<br><code>milestones JSONB [{title, amount, description, duration}]</code> ,<br><code>status</code> ; <code>UNIQUE(project_id, freelancer_id)</code>                                                                                                                                                                                      | Yes   |
| <code>contracts</code>             | The agreement<br>created when a<br>bid is accepted                       | <code>project_id</code> (unique), <code>proposal_id</code> , <code>client_id</code> , <code>freelancer_id</code> ,<br><code>total_amount</code> , <code>locked_amount</code> , <code>released_amount</code> , <code>status</code> ,<br><code>milestones JSONB</code> , <code>client_signature</code> , <code>freelancer_signature</code> ,<br><code>smart_contract_address</code> (always <code>INTERNAL_WALLET</code> ),<br><code>feedback_pending</code> | Yes   |
| <code>milestone_submissions</code> | Proof of work per<br>milestone index<br>— <b>no SQL in repo</b>          | <code>contract_id</code> , <code>milestone_index</code> (needs unique pair),<br><code>proof_of_work_url</code> , <code>description</code> , <code>status</code><br>pending/approved/revision_requested, <code>feedback</code>                                                                                                                                                                                                                              | Yes   |
| <code>contract_messages</code>     | Chat on a<br>contract                                                    | <code>contract_id</code> , <code>sender_id</code> (→ <code>auth.users</code> ), <code>message</code>                                                                                                                                                                                                                                                                                                                                                       | Yes   |

| Table                                                                                                                                           | Purpose                    | Key columns | Used?  |
|-------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|-------------|--------|
| <code>milestones</code> ,<br><code>project_attachments</code> ,<br><code>services</code> ,<br><code>bookmarks</code> ,<br><code>messages</code> | Defined for a richer model | —           | Unused |

## Money

| Table                            | Purpose                                                                           | Key columns                                                                                                                                                                                       | Used?         |
|----------------------------------|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| <code>user_wallets</code>        | Platform token balance, one per user                                              | <code>user_id</code> (unique), <code>token_balance</code> INT CHECK ≥ 0                                                                                                                           | Yes           |
| <code>token_transactions</code>  | Token ledger                                                                      | <code>type</code> BUY/SPEND/RECEIVE/REDEEM, <code>tokens</code> ,<br><code>amount_inr</code> , <code>status</code> PENDING/SUCCESS/FAILED,<br><code>payment_ref</code> , <code>description</code> | Yes           |
| <code>transactions</code>        | Legacy ₹ ledger behind <code>users.balance</code>                                 | <code>type</code> credit/debit, <code>category</code> , <code>status</code> ,<br><code>payment_method</code>                                                                                      | Release only  |
| <code>credit_transactions</code> | Reputation credits                                                                | <code>amount</code> , <code>reason</code>                                                                                                                                                         | Reviews       |
| <code>blockchain_escrows</code>  | Intended link between a project and its on-chain escrow (defaults MATIC / mumbai) | <code>contract_address</code> , <code>tx_hash</code> , wallets, <code>status</code>                                                                                                               | Never written |

## Disputes and arbitration

| Table                                                                             | Purpose                                                                                                                                                                                                                                                                                                          | Used?                                      |
|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| <code>disputes</code>                                                             | Dispute record; TEXT id <code>DSP-</code> + 6 digits; status OPEN/AWAITING_EVIDENCE/UNDER_REVIEW/RESOLVED/ESCALATED; outcome FREELANCER/CLIENT/PARTIAL. Code also uses <code>arbitrator_id</code> , <code>assigned_arbitrator_id</code> , <code>milestone_index</code> , <code>domain</code> — none exist in SQL | Partial                                    |
| <code>dispute_messages</code> (no SQL),<br><code>dispute_evidence</code> (no RLS) | Case chat and evidence                                                                                                                                                                                                                                                                                           | Chat used by the arbitrator room           |
| <code>arbitrator_gigs</code> ,<br><code>arbitrators</code> (no SQL)               | Bookable arbitrator time slots and profiles                                                                                                                                                                                                                                                                      | Gigs used by <code>/resolution-gigs</code> |
| <code>ai_dispute_recommendations</code>                                           | Output of the LangChain agent                                                                                                                                                                                                                                                                                    | Unused                                     |

## Reputation, profile and AI

| Table                                                                                            | Purpose                                                                    | Used?                   |
|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|-------------------------|
| <code>feedback</code>                                                                            | Generic review (reviewer → reviewee, 1–5) — the one actually used          | Yes                     |
| <code>freelancer_feedback</code> , <code>client_feedback</code> , <code>platform_feedback</code> | Detailed two-way feedback with a rating trigger                            | No UI caller            |
| <code>site_reviews</code> (no SQL)                                                               | Platform rating captured with each review                                  | Written, errors ignored |
| <code>user_education</code> , <code>user_certifications</code> , <code>portfolio_items</code>    | Rich profile sections                                                      | Yes                     |
| <code>notifications</code>                                                                       | In-app notifications, realtime-subscribed                                  | Yes                     |
| <code>ai_project_risk_reports</code>                                                             | Cached Gemini risk report per (project, freelancer); no FKs, no unique key | Yes                     |

## 5.4 Row-level security

All policies apply to both anonymous and authenticated requests (no `T0` clause), and the service role bypasses them.

| Table                                                            | Read                          | Write                                                     | Problem                                                                                                                                              |
|------------------------------------------------------------------|-------------------------------|-----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>users</code>                                               | Everyone, including anonymous | Own row, <b>any column</b>                                | Emails, phones and balances are public; users can set their own <code>balance</code> , <code>rating</code> , <code>role</code> , <code>tokens</code> |
| <code>projects</code>                                            | Public or own                 | Owner only                                                | A hired freelancer cannot see a private project                                                                                                      |
| <code>proposals</code>                                           | Freelancer or project owner   | Freelancer inserts (any status); owner updates any column | A bid can be inserted already "accepted"                                                                                                             |
| <code>contracts</code>                                           | Parties                       | <b>Either party inserts or updates any column</b>         | Milestone amounts can be rewritten before release                                                                                                    |
| <code>user_wallets</code> , <code>token_transactions</code>      | Own                           | <b>No write policy</b>                                    | Wallet routes cannot work under the repo's RLS                                                                                                       |
| <code>notifications</code>                                       | Own                           | <b>No insert policy</b>                                   | Cross-user notifications fail silently                                                                                                               |
| <code>disputes</code>                                            | Raiser or parties             | Insert by anyone as raiser; <b>no update</b>              | Arbitrators cannot resolve                                                                                                                           |
| <code>feedback</code>                                            | Everyone                      | Reviewer = self                                           | Anyone can review anyone                                                                                                                             |
| <code>ai_project_risk_reports</code>                             | Own                           | <b>Anyone ( <code>WITH CHECK (true)</code> )</b>          | Reports can be forged for other users                                                                                                                |
| <code>project_attachments</code> , <code>dispute_evidence</code> | —                             | —                                                         | <b>RLS not enabled at all</b>                                                                                                                        |

## 5.5 Functions, triggers, storage and realtime

| Object                                                                                                    | Behaviour                                                                                             | Issue                                                 |
|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| <code>update_updated_at()</code>                                                                          | Sets <code>updated_at</code> on update for most tables                                                | —                                                     |
| <code>update_user_balance()</code> on <code>transactions</code>                                           | Adds/subtracts <code>amount</code> to <code>users.balance</code> when a row is <code>completed</code> | Re-applies on every update of a completed row         |
| <code>handle_new_user_wallet()</code> on <code>public.users</code> insert                                 | Creates the user's wallet (SECURITY DEFINER)                                                          | No <code>search_path</code> ; misleading trigger name |
| <code>update_freelancer_rating()</code>                                                                   | Recomputes rating from <code>freelancer_feedback</code>                                               | Silent no-op under RLS (updates another user's row)   |
| <code>has_submitted_feedback</code> , <code>get_freelancer_rating</code> , <code>get_client_rating</code> | Helper RPCs                                                                                           | Never called                                          |
| Storage bucket <code>avatars</code>                                                                       | Avatar uploads from the profile editor                                                                | Bucket and policies not in SQL                        |
| Realtime                                                                                                  | <code>notifications</code> inserts (header bell); <code>transactions</code> (wallet page)             | Publication not in SQL                                |

## 5.6 State machines

| Entity               | Statuses                                       | Transitions the code performs                                                                                                                                                        |
|----------------------|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Project              | draft, open, in_progress, completed, cancelled | <code>open</code> on posting. Bid accept tries to set <code>in_progress</code> but writes a non-existent column, so <b>projects stay open</b> . Nothing sets completed or cancelled. |
| Proposal             | pending, accepted, rejected, withdrawn         | <code>pending</code> on bid; <code>accepted</code> / <code>rejected</code> on accept. Withdraw not implemented.                                                                      |
| Contract             | active, completed, disputed, cancelled, paused | <code>active</code> on accept. <code>disputed</code> only from an uncalled route. <b>Never completed</b> .                                                                           |
| Milestone submission | pending, revision_requested, approved          | Submit → pending; feedback → revision_requested; release → approved — with no guard against releasing twice.                                                                         |
| Dispute              | OPEN ... RESOLVED                              | Creation routes are uncalled or fail; resolve sets RESOLVED and calls undefined RPCs.                                                                                                |
| Token transaction    | PENDING, SUCCESS, FAILED                       | BUY/SPEND/RECEIVE are SUCCESS immediately; REDEEM stays PENDING forever.                                                                                                             |

## 5.7 Data-model issues to fix first

1. **Capture the live schema** ( `supabase db pull` ) and commit it as ordered migrations.
2. **Collapse duplicated models**: three milestone representations, two review systems, `users` vs `profiles`, five value stores, two wallet DDL copies.
3. **Lock down `users`**: expose a `public_profiles` view; make balance, rating, role, tokens and wallet columns writable only by the server.
4. **Move money into SQL**: atomic `SECURITY DEFINER` functions with row locks for buy, redeem, accept, release and dispute resolution.

5. **Add missing constraints and indexes:** FK `users.id → auth.users`, unique `(contract_id, milestone_index)`, unique risk-report key, `disputes.contract_id`, and indexes on `contracts(client_id)`, `contracts(freelancer_id)` and the timeline queries.
6. **Normalise naming:** one casing for status enums, `hired_freelancer_id` vs `freelancer_id`, job vs project vs bid vs proposal.

## 6 API reference

### 6.1 Conventions

- All routes live under `src/app/api/` as Next.js route handlers; there are **37 handlers in 32 files**.
- Authenticated routes call `supabase.auth.getUser()` (verified server-side) and then read/write **as the user**, so row-level security applies. Only the milestone-release route also uses the service-role client.
- Errors are `{ "error": string }`, usually with the raw PostgREST message. There is no schema validation library in use (zod is installed but unused), no pagination and no rate limiting.
- Three routes wrongly use the **browser** Supabase client on the server (`arbitrator/apply`, `arbitrator/gigs/[gigId]/cancel`, `projects`); they run with no user and do no auth check.
- No API route talks to the blockchain.

### 6.2 Endpoint summary

| Method     | Path                                                      | Auth            | Purpose                                      | Called from UI                    |
|------------|-----------------------------------------------------------|-----------------|----------------------------------------------|-----------------------------------|
| GET        | <code>/api/jobs?status&amp;category&amp;created_by</code> | none            | List projects (with client)                  | dashboard, find-jobs, my-projects |
| POST       | <code>/api/jobs</code>                                    | user            | Create a project (status <code>open</code> ) | post-project                      |
| GET        | <code>/api/jobs/[jobId]</code>                            | none            | One project                                  | job page                          |
| PATCH      | <code>/api/jobs/[jobId]</code>                            | owner           | Update — <b>writes the raw body</b>          | —                                 |
| GET        | <code>/api/jobs/[jobId]/bids</code>                       | owner           | Bids on a project                            | job page (client view)            |
| POST       | <code>/api/bids</code>                                    | user, not owner | Place a bid with milestones                  | job page                          |
| GET        | <code>/api/bids/my-bids?freelancer_id</code>              | none (RLS)      | A freelancer's bids                          | job page, my-projects             |
| POST       | <code>/api/bids/[bidId]/accept</code>                     | project owner   | Accept bid, debit tokens, create contract    | job page                          |
| GET        | <code>/api/contracts?user_id</code>                       | none (RLS)      | Contracts where user is a party              | contracts list                    |
| GET        | <code>/api/contracts/[id]</code>                          | party           | Contract + project + users + submissions     | contract page                     |
| PATCH      | <code>/api/contracts/[id]/sign</code>                     | party           | Store the caller's drawn signature           | contract page                     |
| GET / POST | <code>/api/contracts/[id]/messages</code>                 | none / user     | Contract chat                                | contract page                     |
| POST       | <code>/api/contracts/[id]/dispute</code>                  | party           | Raise a dispute, auto-assign arbitrator      | (call commented out)              |
| POST       | <code>/api/contracts/[id]/milestones/[i]/submit</code>    | freelancer      | Submit proof of work                         | contract page                     |

| Method     | Path                                                     | Auth                | Purpose                                   | Called from UI     |
|------------|----------------------------------------------------------|---------------------|-------------------------------------------|--------------------|
| POST       | <code>/api/contracts/[id]/milestones/[i]/feedback</code> | client              | Request a revision                        | contract page      |
| POST       | <code>/api/contracts/[id]/milestones/[i]/release</code>  | client              | Release payment for a milestone           | contract page      |
| GET / POST | <code>/api/disputes</code>                               | none / user         | List / create disputes                    | —                  |
| GET        | <code>/api/disputes/[id]</code>                          | none (RLS)          | Dispute detail with contract milestone    | arbitrator room    |
| GET / POST | <code>/api/disputes/[id]/messages</code>                 | none / user         | Dispute chat                              | arbitrator room    |
| POST       | <code>/api/disputes/[id]/resolve</code>                  | assigned arbitrator | Decide RELEASE or REFUND                  | arbitrator room    |
| POST       | <code>/api/arbitrator/apply</code>                       | <b>none</b>         | Create arbitrator profile for any user    | —                  |
| POST       | <code>/api/arbitrator/status</code>                      | user                | Go online / offline                       | resolution-gigs    |
| GET        | <code>/api/arbitrator/profile</code>                     | user                | Profile, upcoming gigs, assigned disputes | resolution-gigs    |
| POST       | <code>/api/arbitrator/gigs/book</code>                   | user, ≥ 3000 tokens | Book a time slot                          | resolution-gigs    |
| POST       | <code>/api/arbitrator/gigs/[gigId]/cancel</code>         | <b>none</b>         | Cancel any gig + penalty                  | —                  |
| POST       | <code>/api/feedback/submit</code>                        | party               | Detailed two-way feedback                 | — (dead modal)     |
| POST       | <code>/api/reviews</code>                                | user                | Review + site rating + credits            | contract page      |
| PUT        | <code>/api/profile/update</code>                         | user                | Update own profile (whitelisted fields)   | profile edit       |
| GET / POST | <code>/api/projects</code>                               | <b>none</b>         | Legacy project list / create (broken)     | dashboard/projects |
| GET        | <code>/api/wallet/balance</code>                         | user                | Token balance and ₹ value                 | wallet pages       |
| POST       | <code>/api/wallet/buy</code>                             | user                | Buy tokens ( <b>mock payment</b> )        | token store        |
| POST       | <code>/api/wallet/redeem</code>                          | user                | Request payout (stays PENDING)            | token redeem       |
| GET        | <code>/api/wallet/transactions</code>                    | user                | Token history                             | profile wallet     |
| POST       | <code>/api/ai/project-risk</code>                        | user                | AI risk report (cached)                   | job page           |

## 6.3 Jobs and bids

**POST /api/jobs** requires `title`, `description`, `category`, `budget_type`, `experience_level`. It stores `budget_range`, `budget_min` and `budget_max` from `budget_amount`, builds `duration` as "start to end" when dates are given, and forces `client_id` = caller and `status` = `open`. It **ignores** the `blockchain_tx`, `escrow_locked` and `temp_id` fields the posting page sends after locking SHM, so nothing links a project to its on-chain deposit.

**POST /api/bids** requires `job_id`, `amount`, `proposal`. It rejects bids on your own job or on non-open jobs, and stores `proposed_budget`, `cover_letter`, the `milestones` array and `estimated_duration` ("N milestones"). Negative amounts are not rejected, and milestone amounts are never checked against the bid total on the server (the UI enforces the sum).

**POST /api/bids/[bidId]/accept** — the "funding" step:

1. Verify the caller owns the project and the project is `open`.
2. Read the client's `user_wallets.token_balance`. **If it is lower than the bid, set it to 1000** (a demo "auto top-up").
3. Deduct the bid amount and record a `SPEND` token transaction.
4. Mark this proposal `accepted` and all others `rejected`; try to set the project `in_progress` (fails silently — wrong column name).
5. Only now check whether a contract already exists (after money has moved).
6. Insert the contract with `total_amount = locked_amount = bid`, the bid's milestones and `smart_contract_address = 'INTERNAL_WALLET'`.
7. Notify the freelancer (fails: invalid notification type and RLS).

Returns `201 { success, contractId }`.

## 6.4 Contracts and milestones

- **Sign** stores the caller's drawn-signature PNG data URL in `client_signature` or `freelancer_signature`. Re-signing overwrites; signatures are not required by any other route.
- **Messages** are plain text in `contract_messages`; the GET has no party check in code (RLS only).
- **Submit** (freelancer) upserts `milestone_submissions` as `pending` — this can turn an approved milestone back to pending.
- **Feedback** (client) sets `revision_requested` with the client's comment.
- **Release** (client):
  1. Amount = `contract.milestones[i].amount` (from the freelancer's bid).
  2. Mark the submission `approved`; decrease `contracts.locked_amount` (can go negative).
  3. Credit the freelancer's `user_wallets` **using the client's session** (blocked by RLS in the repo schema).
  4. With the service role: insert a `RECEIVE` token transaction, a completed `transactions` credit (which raises `users.balance` via trigger) and two notifications.
  5. **No checks** that a submission exists, that the milestone was not already released, that the contract is not disputed, or that enough is locked.

## 6.5 Disputes and arbitration

- **Raise (contract route)** builds a dispute with id `DSP-` + last six digits of `Date.now()`, assigns the first user whose `arbitrator_status` is online (falling back to the user named `judge_bot`), and sets the contract to `disputed`. It writes `status: 'pd'` and three columns that are not in the schema, and the UI call is commented out.
- **Raise ( POST /api/disputes )** assigns an arbitrator who has a *booked* gig covering the current time; no code ever books a gig, and no UI calls this route.
- **Resolve** requires `disputes.arbitrator_id = caller`, sets `RESOLVED` with outcome FREELANCER (for `RELEASE`) or CLIENT, then calls `release_escrow_to_freelancer` / `refund_escrow_to_client` RPCs that are not defined, ignoring the error.

- **Arbitrator booking** checks `users.tokens ≥ 3000` and overlap with your own gigs, then inserts an `available` gig. Start/end ordering, future times and the one-hour advance rule are not validated.

## 6.6 Wallet

| Route                          | Behaviour                                                                                                                                                                                                                                                                                  |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>GET /balance</code>      | <code>{ tokenBalance, inrValue: tokens × 10 }</code>                                                                                                                                                                                                                                       |
| <code>POST /buy</code>         | Packages <code>pack_10</code> / <code>pack_50</code> / <code>pack_100</code> (₹100/500/1000) or a custom amount with <b>no upper bound</b> ; inserts <code>BUY / SUCCESS</code> with <code>payment_ref: MOCK_PAY_&lt;ts&gt;</code> and adds to the balance. No payment provider is called. |
| <code>POST /redeem</code>      | Checks the balance, inserts <code>REDEEM / PENDING</code> with the bank account or UPI id in the description, and deducts immediately. Nothing ever processes the request.                                                                                                                 |
| <code>GET /transactions</code> | The caller's token history, newest first                                                                                                                                                                                                                                                   |

All balance changes are read-then-write in application code, so concurrent requests can double-spend.

## 6.7 Reviews, feedback and profile

- `POST /api/reviews` inserts a `feedback` row and a `site_reviews` row. If the caller owns the project it also creates a `portfolio_items` entry for the reviewee and adds +30 `total_credits` and +1 `projects_completed`. It does not check that either person was part of the project.
- `POST /api/feedback/submit` implements the richer two-way model ( `freelancer_feedback` / `client_feedback` / `platform_feedback` ), once per contract per side — but no mounted component calls it.
- `PUT /api/profile/update` whitelists name, profession, about ( $\leq 1000$  chars), skills, languages, location, hourly rate, company and phone.

## 6.8 Business rules: where they are enforced

| Rule                                                          | Server                | Browser only          |
|---------------------------------------------------------------|-----------------------|-----------------------|
| Only the owner sees bids and accepts one                      | ✓                     |                       |
| Cannot bid on your own job; job must be open                  | ✓                     |                       |
| One bid per freelancer per project                            | ✓ (unique constraint) |                       |
| Milestone amounts sum to the bid                              |                       | ✓                     |
| Both parties sign before work or release                      |                       | ✓ (blocking overlay)  |
| Release only after a pending submission; never twice          |                       | ✓                     |
| Disputes freeze payment                                       | — nowhere             |                       |
| Wallet linked once, permanently, with proof                   |                       | ✓ (no proof anywhere) |
| Arbitrator must hold 3000 tokens to go online                 |                       | ✓ (booking checks it) |
| Arbitrator booking $\geq 1$ hour ahead, max 3 specialisations | — nowhere             |                       |
| Post-project form validation (lengths, dates, budget)         | partly                | ✓                     |

## 7 Blockchain, tokens and payments

### 7.1 Smart contracts

All contracts are Solidity `^0.8.20`, compiled by Hardhat with the optimizer at 200 runs. Only `backend/contracts/` is compiled; `blockchain/contracts/` holds older copies ( `Escrow.sol` is byte-identical; its `TrustToken.sol` is a cut-down fixed-supply variant). There are **no contract tests** — `npm test` points at a `test/` folder that does not exist.

#### ProjectEscrow — the full design ( `backend/contracts/ProjectEscrow.sol` , 311 lines)

Milestone escrow in the chain's native coin with admin arbitration. Uses OpenZeppelin `ReentrancyGuard` and `Ownable`.

| Function                                                                                                      | Who            | What it does                                                                                                      |
|---------------------------------------------------------------------------------------------------------------|----------------|-------------------------------------------------------------------------------------------------------------------|
| <code>createEscrow(projectId, freelancer, amounts[]) payable</code>                                           | client         | Requires <code>msg.value</code> = sum of milestones; stores the escrow as FUNDED                                  |
| <code>approveMilestone(projectId, index)</code>                                                               | client         | Marks the milestone approved and <b>pays the freelancer immediately</b> ; COMPLETED when all are paid             |
| <code>raiseDispute(projectId)</code>                                                                          | either party   | Freezes approvals (status DISPUTED)                                                                               |
| <code>resolveDispute(projectId, outcome, pct)</code>                                                          | platform admin | FREELANCER → remaining to freelancer; CLIENT → refund; PARTIAL → <code>pct</code> % to freelancer, rest to client |
| <code>getEscrow</code> , <code>getMilestones</code> , <code>updatePlatformAdmin</code> , <code>version</code> | —              | Views and admin rotation                                                                                          |

Reentrancy handling is sound (value-moving functions are `nonReentrant` ). Weaknesses: project ids are caller-chosen and can be squatted, arbitration is a single admin key, there is no cancel/deadline/auto-release, and no platform fee.

#### This is the contract the product needs

`ProjectEscrow` already encodes the TrustLance promise — per-milestone locking, payment on approval, disputes that freeze funds and a percentage split decided by an arbiter — and its `resolveDispute` signature matches the output of the LangChain dispute agent. It is deployed on Polygon Amoy but **no frontend code references it**.

#### Escrow — what the app actually uses ( `backend/contracts/Escrow.sol` , 54 lines)

A native-SHM deposit box for Shardeum.

- `lockFunds(string projectId) payable` — records `{ amount, client, active }` , emits `FundsLocked` .
- `emergencyWithdraw()` onlyOwner — sends the **entire contract balance to the owner**.
- `getBalance()` view.

There is **no release, no refund, no freelancer address and no dispute logic**; `FundsReleased` is declared but never emitted, and projects never become inactive.

## TrustToken (TRT)

A hand-written ERC-20 ("TrustLance Token", 1,000,000 minted to the deployer, owner-only uncapped `mint`). Its only deployment attempt, on Shardeum, failed with "provided fee < minimum global fee" (`backend/deploy_success.txt` — the file name is misleading). The frontend's TRT address is `0x000...000` ("DUMMY - TRT removed"), and the ABI in `src/lib/contracts/token-contract.ts` (`lockTokens`, `releaseTokens`, ...) matches **no contract in the repo**.

## 7.2 Networks and deployments

| Network                       | Chain id | Configured in Hardhat                  | Deployed contract          | Address                                                 |
|-------------------------------|----------|----------------------------------------|----------------------------|---------------------------------------------------------|
| Polygon Amoy (testnet)        | 80002    | ✓<br>( <code>POLYGON_AMOY_RPC</code> ) | ProjectEscrow, 23 Jan 2026 | <code>0x2DC618147ee8360CD83d5ed3368525B920F2712F</code> |
| Shardeum (labelled "Mainnet") | 8118     | ✓ (RPC hard-coded)                     | Escrow, 14 Feb 2026        | <code>0x2DC618147ee8360CD83d5ed3368525B920F2712F</code> |
| Sepolia                       | 11155111 | ✓                                      | none                       | —                                                       |
| Polygon mainnet               | 137      | ✓                                      | none                       | —                                                       |

The two deployments share one address because both were the deployer's first transaction (`0xD62C...2A79`, nonce 0) on their chain — only the chain id tells them apart. The frontend uses hard-coded values in `src/lib/config.ts`: chain `8118`, escrow `0x2DC6...712F`. The `NEXT_PUBLIC_ESCROW_CONTRACT_ADDRESS` / `NEXT_PUBLIC_CHAIN_ID` variables printed by the deploy script are never read. Explorer links point to the legacy Sphinx testnet explorer, and some UI labels say "Shardeum Sphinx" while the config says mainnet.

Deployment scripts: `scripts/deploy.js` deploys ProjectEscrow (`npm run deploy:amoy|polygon|local`); `scripts/deploy_shardeum.js` deploys Escrow and has **no npm script**.

## 7.3 The platform token

"Tokens" are **integers in Supabase**, not an on-chain asset: `user_wallets.token_balance`, with every movement recorded in `token_transactions`. The conversion is fixed in code at **1 token = ₹10**.

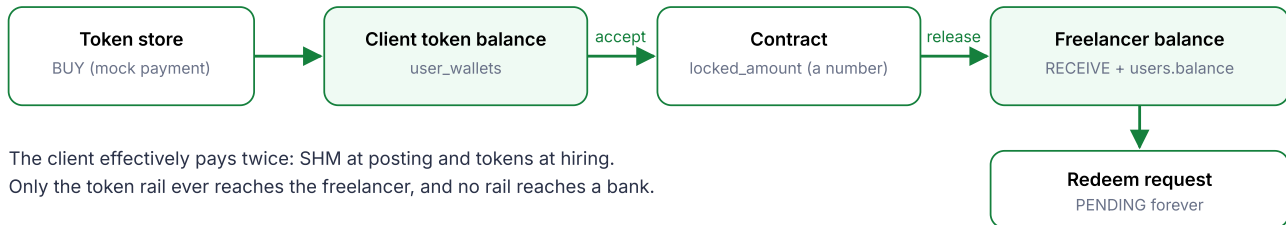
| Operation           | Route                                                        | Real money?                                |
|---------------------|--------------------------------------------------------------|--------------------------------------------|
| Buy                 | <code>POST /api/wallet/buy</code>                            | No — mock payment, credited instantly      |
| Spend (hire)        | <code>POST /api/bids/[id]/accept</code>                      | No — debit; auto top-up to 1000 if short   |
| Receive (milestone) | <code>POST /api/contracts/[id]/milestones/[i]/release</code> | No — credit                                |
| Redeem              | <code>POST /api/wallet/redeem</code>                         | No — request stays PENDING; no payout rail |

## 7.4 End-to-end money flow

### On-chain (SHM, Shardeum 8118)



### Off-chain (platform tokens in Supabase, 1 token = ₹10)



The client effectively pays twice: SHM at posting and tokens at hiring.  
Only the token rail ever reaches the freelancer, and no rail reaches a bank.

Figure 7.1 — The two money rails as implemented

| Step                                       | Code                                                                          | Nature                                                                         |
|--------------------------------------------|-------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| 1. Client buys tokens                      | token store → <code>/api/wallet/buy</code>                                    | DB only, mock payment                                                          |
| 2. Client posts project and "locks escrow" | <code>post-project/page.tsx:453-514</code><br>→ <code>Escrow.lockFunds</code> | <b>Real SHM</b> to a contract only the owner can empty; not recorded in the DB |
| 3. Client accepts a bid                    | <code>/api/bids/[id]/accept</code>                                            | DB only; tokens debited, contract created                                      |
| 4. Both sign                               | <code>/api/contracts/[id]/sign</code>                                         | DB only; drawn image, not cryptographic                                        |
| 5. Freelancer submits work                 | <code>.../submit</code>                                                       | DB only                                                                        |
| 6. Client releases a milestone             | <code>.../release</code>                                                      | DB only; repeatable                                                            |
| 7. Freelancer cashes out                   | <code>/api/wallet/redeem</code>                                               | PENDING request; no payout                                                     |
| Refunds                                    | —                                                                             | Not implemented anywhere                                                       |
| Disputes                                   | <code>/api/disputes/[id]/resolve</code>                                       | Calls undefined RPCs; moves nothing                                            |
| Platform fee                               | —                                                                             | None (UI copy mentions 10–15%)                                                 |

## 7.5 Wallet integration

- **Provider:** `WalletProvider` ( `src/contexts/wallet-context.tsx` ) wraps the app; it uses ethers v6 `BrowserProvider` over `window.ethereum`.
- **Connect:** `eth_requestAccounts`, then `personal_sign` of a random message (discarded), then a mock local "user" ("John Doe") stored in `localStorage`. Without MetaMask it pretends to connect a hard-coded demo address.
- **Network:** "Switch to Shardeum" calls `wallet_switchEthereumChain` to `0x1fb6` (8118) and adds the chain if missing. Posting a project only alerts on the wrong chain.
- **Balance:** native balance of whatever chain is active, labelled SHM, polled every 30 s. "Locked in escrow" is hard-coded to `0.0`.
- **Gaps:** no `accountsChanged` handler (the UI can show a stale account), a stale `chainChanged` closure, and the `/wallet` page queries transactions with the mock user id so it never finds any.

## 7.6 Payments that are simulated

| Feature                                                                          | Reality                                                                                                                |
|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| PayPal connect ( <code>/profile/payments/paypal</code> )                         | A 2-second <code>setTimeout</code> , then "success"; the status page still shows "Not connected"                       |
| <code>PaymentManager</code> ( <code>src/lib/payments/payment-manager.ts</code> ) | <code>console.log</code> stubs; the ERC-20 path targets token address <code>0x0</code> and has an invalid ABI fragment |
| Fund management, transaction history, financial dashboard modals                 | Hard-coded data and <code>alert()</code>                                                                               |
| Stripe                                                                           | Listed as a payment method type; no provider                                                                           |

## 7.7 Security of funds — summary

1. Client SHM is custodied by a single developer key with no release path.
2. Value can be created for free (mock buy, auto top-up).
3. Release is repeatable and its amount can be edited by either party under RLS.
4. Balance updates are non-atomic read-modify-write.
5. A failed `/api/jobs` call after a successful lock strands the deposit; retrying locks again.
6. The same deployer key is configured for testnets **and** mainnets (Polygon 137, Shardeum 8118), although the setup guide calls it a testnet-only key.

Chapter 14 rates each issue and chapter 16 describes the fix: move to `ProjectEscrow`-style per-milestone escrow (or a single, properly reconciled off-chain ledger) and remove every simulated path before real money is involved.

## 8 AI features

### 8.1 Inventory

TrustLance's UI shows "AI" in many places, but only one feature calls a real model.

| # | Feature                                  | Engine                                                        | Real model call | Reachable                                       |
|---|------------------------------------------|---------------------------------------------------------------|-----------------|-------------------------------------------------|
| A | <b>Project Risk Analysis</b>             | Genkit + Gemini 2.0 Flash                                     | Yes             | Yes — job page, "Apply to this Job"             |
| B | AI project recommendations               | Genkit + Gemini                                               | Would be        | No — only in the Genkit dev UI                  |
| C | Dispute resolution agent                 | LangChain JS (OpenAI gpt-3.5-turbo / Ollama llama3:8b / mock) | Would be        | No — nothing imports it                         |
| D | "AI Dispute Assistant" chat              | Keyword <code>if/else</code> + 1.5 s delay                    | No              | No — only in the dead <code>page_old.tsx</code> |
| E | "AI pre-analysis" when opening a dispute | <code>setTimeout</code> + <code>Math.random</code>            | No              | Page broken by merge conflict                   |
| F | Dispute "AI verdict"                     | Hard-coded objects                                            | No              | Yes, mock data                                  |
| G | Admin "AI accuracy / bias" metrics       | Constants (87.3 %, 12.5 % ...)                                | No              | Yes, public <code>/admin/disputes</code>        |
| H | Bid Insights "PRO Intelligence"          | Static numbers                                                | No              | Yes, profile menu                               |

### 8.2 A. Project Risk Analysis

**Purpose.** Before bidding, a freelancer sees an objective assessment of the project's risk: vague scope, unrealistic timeline, budget problems, missing milestones and likelihood of disputes.

**Trigger.** A signed-in user who is not the project owner clicks **Apply to this Job** on `/job/[jobId]`. The modal posts `{ projectId }` to `/api/ai/project-risk`; "Proceed Anyway" then opens the bid form. The report can be reopened later.

**Pipeline.**

- `POST /api/ai/project-risk` authenticates the user and loads the project.
- Cache:** if `ai_project_risk_reports` has a row for (project, user), return it.
- Map project fields to the model input: `title`, `description`, `budget` (from `budget_min`), `duration` and `milestones` — the last one reads a column that does not exist, so the model is always told "None".
- Run the Genkit flow `projectRiskAnalysisFlow` (`src/ai/flows/project-risk-analysis.ts`) with the model `googleai/gemini-2.0-flash` (`src/ai/genkit.ts`).
- Save the result and return it.

**Prompt (summary).** "You are an AI assistant analysing freelance projects for potential risk, objectively and neutrally." It lists the five checks above, inserts the project fields, and demands "ONLY valid JSON" in a fixed shape, with no advice, legal terms or extra text.

**Schemas.**

```
input: { title: string, description: string, budget: string|number,
        duration: string, milestones?: string }
output: { risk_level: "LOW" | "MEDIUM" | "HIGH",
         risk_summary: string, issues: string[], confidence_score: number }
```

**Presentation.** `RiskAnalysisModal` shows a colour-coded risk level, the issue list and a confidence bar, plus a disclaimer. A PDF of the report (jsPDF) **downloads automatically** every time the modal opens.

**Cost and latency.** One Gemini Flash call per (project, freelancer) pair, a few hundred tokens each way — a small fraction of a cent per report, a few seconds of latency on the request path, with no timeout, retry or rate limit.

#### Problems to fix before relying on it

**Fake reports.** Any error (missing key, quota, schema mismatch) produces a hard-coded "LOW risk, 85 % confidence" report that is **saved as if real** and then served from cache forever for that user.

**Forgeable cache.** The table's insert policy is `WITH CHECK (true)`, so anyone can insert a report for another user; with no unique key, duplicates also break the cache lookup and trigger repeated model calls.

**Prompt injection.** The client writes the description that goes straight into the prompt and could steer the verdict.

**Missing context.** Milestones, skills, category, experience level and currency are not passed to the model.

## 8.3 B. AI project recommendations (unused)

`src/ai/flows/ai-project-recommendations.ts` is the Firebase Studio template flow: given a free-text user profile it asks Gemini for N recommended projects or freelancers with a match score from 0 to 1. It has no database grounding — the model would invent ids and names — and no caller.

## 8.4 C. LangChain dispute agent (unused)

`backend/ai/dispute-agent.js` (227 lines) was designed to recommend dispute outcomes:

- **Provider** chosen by `AI_PROVIDER`: `openai` (`MODEL_NAME`, default gpt-3.5-turbo), `ollama` (llama3:8b at `OLLAMA_BASE_URL`, the default) or `mock`.
- **Data:** reads the dispute, the project with its milestones, messages and evidence using the service-role key.
- **Output** (zod schema): `outcome` FREELANCER / CLIENT / PARTIAL, `freelancer_percentage`, `confidence_score`, `reasoning`, `key_factors`, `evidence_summary` — exactly the inputs of `ProjectEscrow.resolveDispute`.
- **Mock mode** decides from the share of approved milestones (> 80 % → freelancer, < 30 % → client, otherwise partial).
- **Gaps:** the gathered messages and evidence never reach the prompt, it reads the unused `milestones` table, it does not load environment variables, and nothing — no route, script or import — runs it. Its dependencies live only in `backend/node_modules`, so the Next.js app cannot import it as-is.

## 8.5 D–H. Simulated AI

These are UI demonstrations without models: keyword-matched chat replies, `Math.random` risk scores, hard-coded verdicts on two sample disputes (DSP-001 and DSP-002), constant admin metrics and static bid insights. They should either be connected to real logic or clearly labelled as demos.

## 8.6 Recommended AI roadmap

1. Make the risk report trustworthy: never persist fallbacks, add a unique key and server-only inserts, pass the real milestones and skills, and wrap the description as untrusted data in the prompt.
2. Cache per project (not per user) so N applicants cost one model call.
3. Turn the dispute agent into a server-side Genkit flow that reads the contract, submissions, chat and evidence, and produces a recommendation the arbitrator sees — never an automatic decision.
4. Replace the simulated screens with the real outputs, or remove them.

## 9 Frontend

### 9.1 Application shell

```
<html>
  <AuthProvider>           session + profile (contexts/auth-context.tsx)
    <WalletProvider>       MetaMask state (contexts/wallet-context.tsx)
      <ConditionalLayout>  chooses the shell for the current route
        <PublicLayout>     Header + page + Footer           (public pages)
        <AuthLayout>       AuthenticatedHeader + page       (signed-in pages)
      <Toaster/>
```

- `ChatProvider` exists but is **never mounted**.
- There are no `loading.tsx`, `error.tsx` or `not-found.tsx` files and no error boundaries.
- Almost every page is a client component that fetches data in `useEffect`; only `/u/[username]`, `/profile`, `/profile/[username]` and `/edit-profile` are server components.

### 9.2 Navigation

- **Public header** (`components/layout/header.tsx`): logo, three hover mega-menus (Hire Freelancers, Find Work, Solutions — every link is `#`), Log In / Sign Up and Post a Project. It shows Log In / Sign Up even to signed-in users and has no mobile menu.
- **Authenticated header** (`components/layout/authenticated-header.tsx`, 1,123 lines): Dashboard, Jobs, Contracts, Resolution Gigs; "Browse" and "Manage" dropdowns (mostly links to pages that do not exist); search (to a missing `/search` page); wallet widget; **real-time notifications bell**; profile menu with membership, analytics, bid insights, wallet, payments, funds, transactions, financial dashboard, support and logout (most of these modals are hard-coded). A secondary row links Lists, Tasklists, My Projects, Services, Inbox, Feedback, Project Updates, Bookmarks and Disputes.

### 9.3 Page inventory

**Working** real data end to end · **Partial** real but with gaps · **Mock** hard-coded · **Broken** does not compile or always fails

| Route                                                                    | Purpose                                             | Data                                      | Status         |
|--------------------------------------------------------------------------|-----------------------------------------------------|-------------------------------------------|----------------|
| <code>/</code>                                                           | Landing page                                        | Static                                    | <b>Working</b> |
| <code>/login</code> , <code>/signup</code> , <code>/auth/callback</code> | Google and email auth                               | Supabase Auth                             | <b>Working</b> |
| <code>/dashboard</code>                                                  | Browse open projects + SHM wallet card              | <code>/api/jobs</code>                    | <b>Working</b> |
| <code>/dashboard/jobs</code>                                             | Job marketplace with filters                        | <code>/api/jobs</code><br>(mock fallback) | <b>Partial</b> |
| <code>/post-project</code>                                               | 9-step posting wizard, signature, on-chain SHM lock | chain + <code>/api/jobs</code>            | <b>Partial</b> |
| <code>/find-jobs</code>                                                  | "Top jobs" listing                                  | <code>/api/jobs</code>                    | <b>Partial</b> |

| Route                                                                                                                                                                                                                                                                                                                   | Purpose                                                                        | Data                                                    | Status                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|---------------------------------------------------------|------------------------|
| <code>/job/[jobId]</code>                                                                                                                                                                                                                                                                                               | Job detail: client sees bids; freelancer sees AI risk report + bid form        | jobs, bids, AI APIs                                     | Working                |
| <code>/my-projects</code>                                                                                                                                                                                                                                                                                               | Posted jobs and my bids                                                        | <code>/api/jobs</code> , <code>/api/bids/my-bids</code> | Working                |
| <code>/contracts</code> , <code>/contracts/[id]</code>                                                                                                                                                                                                                                                                  | Contract list and workspace (sign, submit, revise, release, chat, review, PDF) | contract APIs                                           | Working                |
| <code>/notifications</code>                                                                                                                                                                                                                                                                                             | Notification centre                                                            | Supabase direct                                         | Working                |
| <code>/profile/wallet</code> , <code>/profile/token-store</code> , <code>/profile/token-redeem</code>                                                                                                                                                                                                                   | Token wallet                                                                   | <code>/api/wallet/*</code>                              | Partial (mock payment) |
| <code>/wallet</code>                                                                                                                                                                                                                                                                                                    | MetaMask wallet, link wallet to account                                        | ethers + Supabase                                       | Partial                |
| <code>/u/[username]</code>                                                                                                                                                                                                                                                                                              | Public profile (overview, portfolio, reviews, projects)                        | Supabase (SSR)                                          | Partial                |
| <code>/profile/edit</code>                                                                                                                                                                                                                                                                                              | Edit profile, avatar, education                                                | Supabase + <code>/api/profile/update</code>             | Broken                 |
| <code>/resolution-gigs</code>                                                                                                                                                                                                                                                                                           | Arbitrator dashboard: eligibility, slots, online toggle                        | arbitrator APIs                                         | Partial                |
| <code>/resolution-gigs/room/[id]</code>                                                                                                                                                                                                                                                                                 | Arbitrator case room: details, chat, verdict                                   | dispute APIs                                            | Working *              |
| <code>/disputes</code> , <code>/disputes/[id]</code>                                                                                                                                                                                                                                                                    | Dispute dashboard and detail                                                   | Hard-coded                                              | Mock                   |
| <code>/disputes/new</code>                                                                                                                                                                                                                                                                                              | 6-step dispute wizard                                                          | Partly Supabase, mock AI                                | Broken                 |
| <code>/admin/disputes</code>                                                                                                                                                                                                                                                                                            | Admin console                                                                  | Hard-coded, unprotected                                 | Mock                   |
| <code>/projects</code> , <code>/projects/[id]</code> , <code>/projects/[id]/workspace</code>                                                                                                                                                                                                                            | Older project views                                                            | Hard-coded                                              | Mock / Broken          |
| <code>/project/[jobId]/bids</code> , <code>/my-applications</code> , <code>/dashboard/projects</code>                                                                                                                                                                                                                   | Alternate flows                                                                | Missing endpoints                                       | Broken                 |
| <code>/inbox</code> , <code>/bookmarks</code> , <code>/lists</code> , <code>/tasklists</code> , <code>/project-updates</code> , <code>/services</code> , <code>/quotes</code> , <code>/groups</code> , <code>/solutions</code> , <code>/prototyper</code> , <code>/payment-demo</code> , <code>/profile/payments</code> | Secondary features                                                             | Local state                                             | Mock                   |
| <code>/feedback</code>                                                                                                                                                                                                                                                                                                  | Community feedback wall                                                        | Supabase direct                                         | Partial                |

\* The arbitrator room works only for disputes created outside the UI.

About **40 links** point to routes that do not exist (for example `/categories/*`, `/browse`, `/search`, `/contests`, `/profile/settings`, `/escrow/{id}`), including the link stored in every "new bid" notification (`/projects/{id}/bids`).

## 9.4 Design system

- **shadcn/ui** (default style, neutral base, CSS variables, lucide icons). About 20 primitives are used; 12 are installed but unused.
- **Theme tokens** in `src/app/globals.css`: primary indigo `#4F46E5`, foreground `#1F2937`, radius 0.5 rem, a full `.dark` palette. Fonts: Inter (body, headlines) and Source Code Pro (code) via `next/font`.
- **In practice** pages hard-code colours ( `bg-white`, `bg-blue-600`, slate headers, purple dispute pages), so the dark-mode toggle (not persisted) affects little.
- Charts use recharts with hard-coded data; PDFs use jsPDF + autotable (agreement, contract, risk report).

## 9.5 State and data

| Concern     | Implementation                                                                                                                                                                                                                                   |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Auth        | <code>AuthContext</code> : <code>isAuthenticated</code> , <code>user</code> (projection of the <code>users</code> row), <code>login</code> , <code>signup</code> , <code>signInWithGoogle</code> , <code>logout</code> , <code>linkWallet</code> |
| Wallet      | <code>WalletContext</code> : MetaMask connection, chain id, SHM balance (30 s poll), mock projects and unused role helpers                                                                                                                       |
| Server data | Per-page <code>useEffect</code> + <code>fetch</code> ; no SWR/React Query                                                                                                                                                                        |
| Real time   | Supabase Realtime for notifications (header) and wallet transactions; polling every 5 s / 3 s in the arbitrator pages; contract chat only refreshes on reload                                                                                    |
| Errors      | Mixed: <code>alert()</code> (11 places), toasts, inline banners and silent <code>console.error</code> ; several pages fall back to <b>fake data on error</b> , hiding real failures                                                              |

## 9.6 Responsiveness and accessibility

- Marketing sections and dashboards use responsive grids, but **there is no mobile navigation** anywhere and the hover-only mega-menus cannot be used on touch screens or with a keyboard.
- One `aria-label` exists in the whole app; dropdowns lack ARIA roles; the profile menu trigger is a non-focusable `div`; many labels are not associated with inputs; the signature pad has no keyboard alternative; critical money confirmations use `confirm()`.
- Radix dialogs, tabs and selects provide focus management where they are used.

## 9.7 Frontend defects that block a build

1. Unresolved merge-conflict markers in `src/app/projects/[id]/page.tsx:50-54` and `src/app/disputes/new/page.tsx:54-171`.
2. Imports of the non-existent `@/components/ui/use-toast` (real path: `@/hooks/use-toast`) in `profile/edit/page.tsx`, `profile/education-list.tsx` and `feedback/feedback-modal.tsx`.
3. `useSearchParams()` without a `<Suspense>` boundary in `disputes/new`.
4. `TabsContent` rendered outside `Tabs` in `profile/edit`.
5. Rules-of-Hooks violations in `authenticated-header.tsx` and `prototyper/page.tsx`.

`next.config.ts` sets `typescript.ignoreBuildErrors` and `eslint.ignoreDuringBuilds`, which hides type errors but cannot hide these parse and module errors.

## 10 End-to-end user journeys

### 10.1 Journey 1 — Joining TrustLance

1. A visitor lands on `/` and clicks **Post a Project** or **Find Work**; both require sign-in and redirect to `/login`.
2. **Google:** `signInWithOAuth` → Google → `/auth/callback` exchanges the code, creates the `users` row with a generated username and `role: 'freelancer'`, and redirects to `/dashboard`. A trigger creates the token wallet (balance 0).
3. **Email:** `signUp` and a browser-side profile insert (fails if email confirmation is on — see chapter 4).
4. There is no onboarding step. The profile can be viewed at `/u/{username}`; editing (`/profile/edit`) is currently broken by a missing import.
5. **Wallet:** on `/wallet`, Connect MetaMask, then **Link Wallet to Account**, which writes `users.wallet_address` from the browser (no ownership proof). "Switch to Shardeum" adds and selects chain 8118.

### 10.2 Journey 2 — From project to payment (the core flow)

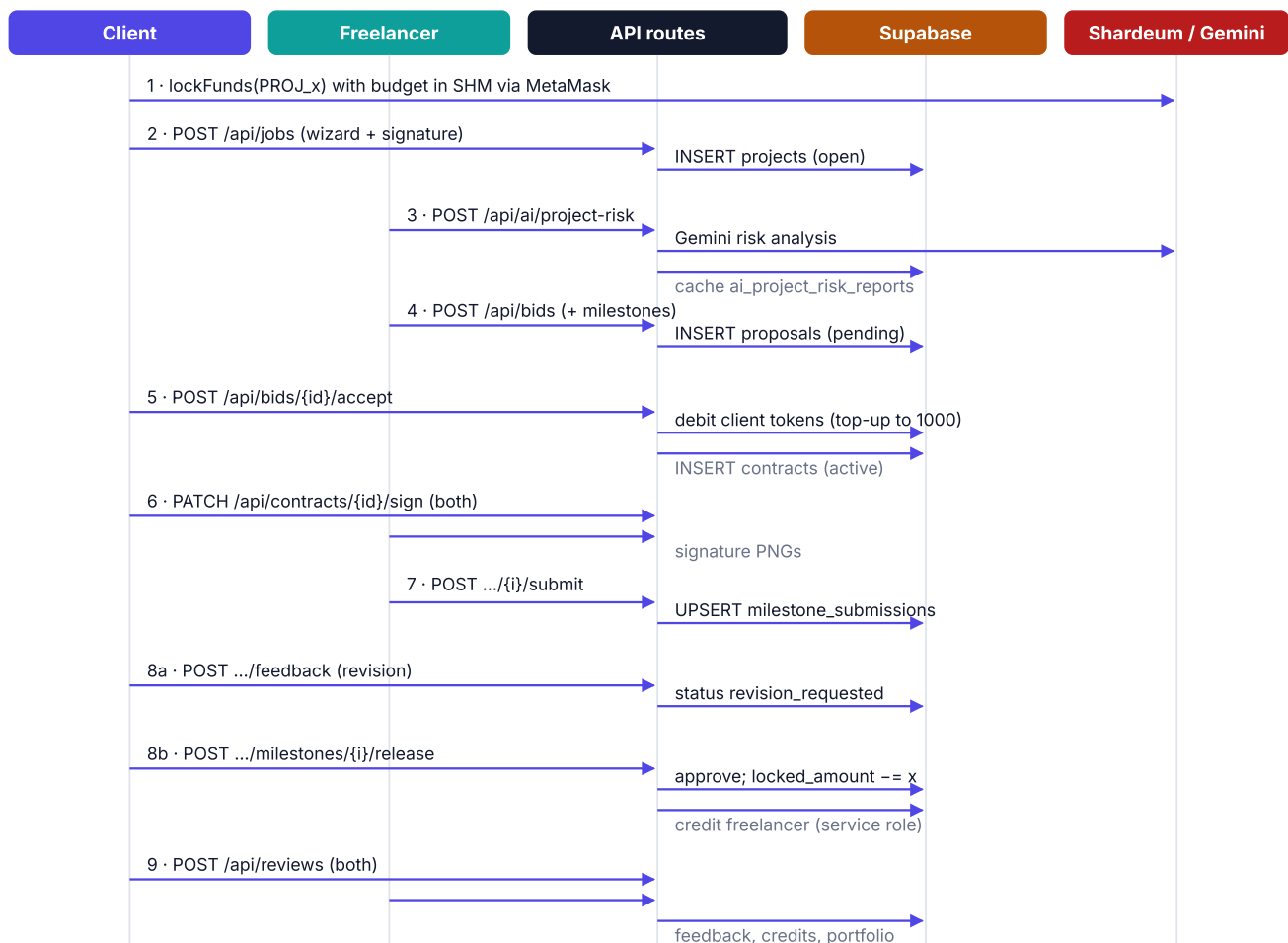


Figure 10.1 — Project lifecycle across actors and services

| #  | Actor      | Screen                                                                           | What happens                                                                                                                                                             | Gaps                                                                             |
|----|------------|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| 1  | Client     | <code>/post-project</code><br>steps 1–7                                          | Title (≥ 10 chars), description (≥ 30), category, subcategory, budget type and amount (SHM), start/end dates, experience level, up to 10 skills, location and visibility | File attachments are collected but never uploaded                                |
| 2  | Client     | Step 8                                                                           | Reads the agreement and draws a signature                                                                                                                                | Step 8 is not validated — a project can be posted unsigned                       |
| 3  | Client     | Step 9 "Post Project"                                                            | MetaMask must be on chain 8118;<br><code>Escrow.lockFunds("PROJ_XXXX", { value: budget })</code> ; then <code>POST /api/jobs</code>                                      | The lock is not linked to the project; if the API fails the SHM is stranded      |
| 4  | Freelancer | <code>/dashboard</code> or <code>/dashboard/jobs</code> → <code>/job/{id}</code> | Browses open projects                                                                                                                                                    | <code>/find-jobs</code> links to a broken page                                   |
| 5  | Freelancer | "Apply to this Job"                                                              | AI risk report in a modal (PDF auto-downloads); "Proceed Anyway"                                                                                                         | Fallback reports are fabricated                                                  |
| 6  | Freelancer | Bid form                                                                         | Amount, duration, milestones whose amounts must sum to the bid; <code>POST /api/bids</code> ; client notified                                                            | Notification link is a 404                                                       |
| 7  | Client     | <code>/my-projects</code> → <code>/job/{id}</code>                               | Sees all bids with an Accept button                                                                                                                                      | Accepted bids still show "Accept"                                                |
| 8  | Client     | "Accept Bid" (no confirmation)                                                   | Tokens debited (auto top-up), contract created, redirect to <code>/contracts/{id}</code>                                                                                 | Project stays <code>open</code> ; double click charges twice                     |
| 9  | Both       | <code>/contracts/{id}</code>                                                     | A blocking "Signature Required" overlay until each party signs the contract                                                                                              | Client signs twice (project and contract); overlay cannot be dismissed           |
| 10 | Freelancer | Milestone card                                                                   | Proof-of-work URL + description → Submit                                                                                                                                 | —                                                                                |
| 11 | Client     | Milestone card                                                                   | "Request revision" with comments, or "Approve & Release" in a confirmation dialog                                                                                        | Release is repeatable server-side; contract never becomes <code>completed</code> |
| 12 | Both       | After the last release                                                           | Feedback modal opens: rate the counterpart and the platform; client's review creates a portfolio item and +30 credits for the freelancer                                 | —                                                                                |
| 13 | Both       | "Download agreement"                                                             | jsPDF agreement with both signatures                                                                                                                                     | Uses the project signature, not the contract one                                 |
| —  | Both       | Contract chat                                                                    | Plain-text messages                                                                                                                                                      | Counterpart's messages appear only after a reload                                |

## 10.3 Journey 3 — Disputes

**As designed** (`DISPUTE_RESOLUTION_SYSTEM.md`): a party opens a dispute in a six-step wizard with evidence; an AI analyses the contract and communications; three anonymous domain experts vote; an admin can override; funds are paid out or refunded automatically with a full audit trail.

### As built:

- "Raise Dispute" on the contract page goes to `/disputes/new` (the direct API call is commented out).
- `/disputes/new` does not compile (merge conflict). Its "AI analysis" is random, evidence upload is cosmetic and **Submit only navigates to** `/disputes` — no dispute is saved.

3. `/disputes` and `/disputes/{id}` show two hard-coded sample disputes; their actions only log to the console.
4. **The real part — arbitration:** a user with  $\geq 3,000$  tokens books 1-hour slots on `/resolution-gigs` (next 7 days, 09:00–21:00), goes online from five minutes before a slot, and is polled every 5 s for assigned disputes. In the room (`/resolution-gigs/room/{id}`) they read the case, chat with 3-second polling and choose **Release** (freelancer) or **Refund** (client).
5. Resolution marks the dispute RESOLVED but the payout RPCs do not exist, so **no balance changes**; the contract stays `disputed`, and release still works during a dispute.
6. The parties have no screen to follow a real dispute, and `/admin/disputes` is a public mock.

## 10.4 Journey 4 — Wallet and tokens

1. **Buy** on `/profile/token-store`: 10 / 50 / 100 tokens for ₹100 / 500 / 1000, or a custom amount; choose UPI, card or net banking (a label only). Tokens are credited instantly by a mock payment.
2. **Balance and history** on `/profile/wallet` (real `token_transactions`). This page is not linked from the main navigation; the header's "Wallet" goes to the MetaMask page `/wallet`.
3. **Redeem** on `/profile/token-redeem` with bank or UPI details: the balance is debited and a PENDING request is recorded that nothing ever pays out.
4. **MetaMask wallet** (`/wallet`): shows the SHM balance; its "Pending / Earnings / Spent" cards are computed from the balance with fixed multipliers, and the transactions list never matches.
5. **PayPal** (`/profile/payments`) is simulated.

## 10.5 Journey 5 — Messaging

- **Contract chat** — the only real messaging: plain text in `contract_messages`, available after hiring, refreshed on load.
- **Dispute chat** — real, arbitrator-facing, polled every 3 s.
- **Inbox and floating chat widget** — hard-coded conversations; the widget is never rendered.
- **XMTP end-to-end encryption** — not present.

## 10.6 Journey 6 — Reputation

Reviews written after a contract appear on `/u/{username}` (rating, text, reviewer) and on the community wall `/feedback`. Client reviews add an automatic portfolio item and 30 "trust credits" to the freelancer. Because every account is created as `freelancer`, the "client reviews" tab on `/feedback` is effectively always empty, and the profile shows a hard-coded 98 % success rate.

# 11 Configuration, environments and running locally

## 11.1 Environment variables

Copy `.env.example` to `.env.local` in the repository root. The Next.js app and Hardhat (run from `backend/`, which loads `../.env.local`) both read it. Exceptions: `npm run genkit:dev` loads `.env`, and the LangChain agent loads nothing.

| Variable                                                                                                        | Used by                         | Purpose                                                   |
|-----------------------------------------------------------------------------------------------------------------|---------------------------------|-----------------------------------------------------------|
| <code>NEXT_PUBLIC_SUPABASE_URL</code>                                                                           | app, middleware, scripts, agent | Supabase project URL                                      |
| <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>                                                                      | app                             | Public anon key (safe to expose; RLS protects data)       |
| <code>SUPABASE_SERVICE_ROLE_KEY</code>                                                                          | release route, scripts, agent   | <b>Secret</b> — bypasses RLS; server only                 |
| <code>GEMINI_API_KEY</code>                                                                                     | Genkit Google AI plugin         | Risk analysis (also accepts <code>GOOGLE_API_KEY</code> ) |
| <code>AI_PROVIDER</code> , <code>OPENAI_API_KEY</code> , <code>OLLAMA_BASE_URL</code> , <code>MODEL_NAME</code> | dispute agent                   | Unused at runtime (agent not wired)                       |
| <code>DEPLOYER_PRIVATE_KEY</code>                                                                               | Hardhat (all networks)          | <b>Secret</b> — deployer wallet key                       |
| <code>PLATFORM_ADMIN_ADDRESS</code>                                                                             | <code>deploy.js</code>          | ProjectEscrow arbiter address                             |
| <code>POLYGON_AMOY_RPC</code> , <code>POLYGON_RPC</code> , <code>SEPOLIA_RPC</code>                             | Hardhat                         | RPC overrides                                             |
| <code>POLYGONSCAN_API_KEY</code>                                                                                | Hardhat verify                  | Contract verification on Polygon                          |
| <code>ANALYZE</code>                                                                                            | <code>next.config.ts</code>     | Enables the bundle analyser                               |

Not configurable (hard-coded in `src/lib/config.ts`): chain id `8118`, escrow address `0x2DC6...712F`, TRT address `0x0`. The Shardeum RPC (`https://api.shardeum.org`) is hard-coded in `hardhat.config.js` and the wallet context.

## 11.2 Scripts and ports

| Command                                         | Where   | What                                                                 |
|-------------------------------------------------|---------|----------------------------------------------------------------------|
| <code>npm run dev</code>                        | root    | Next.js dev server with Turbopack on <b>port 9002</b>                |
| <code>npm run build</code> / <code>start</code> | root    | Production build (POSIX env syntax) / serve on port 3000             |
| <code>npm run typecheck</code>                  | root    | <code>tsc --noEmit</code>                                            |
| <code>npm run genkit:dev</code>                 | root    | Genkit developer UI (registers only the unused recommendations flow) |
| <code>npm run lint</code>                       | root    | <code>next lint</code> — no ESLint config or package exists          |
| <code>npm run compile</code>                    | backend | Compile contracts                                                    |

| Command                                                                                    | Where   | What                                                  |
|--------------------------------------------------------------------------------------------|---------|-------------------------------------------------------|
| <code>npm run deploy:amoy</code> / <code>deploy:polygon</code> / <code>deploy:local</code> | backend | Deploy <b>ProjectEscrow</b>                           |
| <code>npx hardhat run scripts/deploy_shardeum.js --network shardereum</code>               | backend | Deploy the <b>Escrow</b> the app uses (no npm script) |
| <code>npx tsx src/scripts/&lt;name&gt;.ts</code>                                           | root    | Demo-user and diagnostic scripts (service role)       |

## 11.3 Build configuration

- `next.config.ts`: `typescript.ignoreBuildErrors` and `eslint.ignoreDuringBuilds` (template defaults — should be turned off), remote images from placeholder.co, unsplash and picsum, `compress`, `poweredByHeader: false`, optimised imports for Radix and lucide, optional webpack bundle analyser.
- `tsconfig.json`: strict mode, `@/* → src/*`, bundler resolution.
- Tailwind 3 with `tailwindcss-animate`; PostCSS without autoprefixer.
- **Hosting templates**: `apphosting.yaml` (Firebase App Hosting, one instance, no environment configured) and `.idx/dev.nix` (Firebase Studio: Node 20, Firebase emulators that the app does not use). Nothing shows the app was ever deployed there.

## 11.4 Setting up a working local environment

The documented setup ( `SETUP_STEPS.md` ) is incomplete. A setup that matches what the code needs:

1. **Install**: Node 20+, then `npm install` in the root and in `backend/`.
2. **Supabase project**: create one and fill `.env.local` from `.env.example`.
3. **Database** — run in this order in the SQL editor:
  - `schema.sql`
  - `rls_policies.sql` (re-save the UTF-16 tail as UTF-8 first, or paste the `contract_messages` block separately)
  - `feedback_schema.sql`
  - migrations `20240126`, `20240130`, `20240214_add_project_date_range`, `20240215_*` (skip `20240214_create_wallet_system` — it duplicates `schema.sql`)
  - then create the six missing tables, the missing columns ( `users.tokens`, `users.arbitrator_status`, `disputes.arbitrator_id`, `milestone_index`, `domain` ) and the escrow RPCs — or, better, pull them from the live project with `supabase db pull`.
4. **Storage**: create a public `avatars` bucket with an upload policy for signed-in users.
5. **Realtime**: add `notifications` (and `transactions`) to the `supabase_realtime` publication.
6. **Auth**: enable Google OAuth with redirect `http://localhost:9002/auth/callback`; disable email confirmation or move profile creation into a database trigger.
7. **AI**: set `GEMINI_API_KEY`.
8. **Blockchain**: install MetaMask, add Shardeum (the app offers "Switch to Shardeum"), fund the wallet with SHM. To use your own escrow, deploy with `deploy_shardeum.js` and update `src/lib/config.ts`.
9. **Fix the build blockers** listed in chapter 15, then `npm run dev` and open `http://localhost:9002`.

## 11.5 Dependency notes

- **Used**: Next 15.5, React 19, Supabase SSR + JS, ethers 6, Genkit + Google AI, jsPDF + autotable, recharts, react-signature-canvas, Radix primitives, date-fns, Tailwind.

- **Installed but unused:** `@genkit-ai/next`, `@hookform/resolvers`, `react-hook-form`, `zod` (direct), `firebase`, `@supabase/auth-helpers-nextjs` (deprecated), `patch-package`, `react-day-picker`, `embla-carousel-react`, and several Radix packages behind unused shadcn wrappers.
- **Missing:** `eslint` and `eslint-config-next` for `npm run lint`.
- **Backend:** Hardhat 2.28, OpenZeppelin 5.4, hardhat-toolbox 4, dotenv; LangChain packages and `openai` only for the unused agent.

## 12 Project history

### 12.1 Timeline

| Phase                | Dates                        | What happened                                                                                                                                                                                                                                                                                               |
|----------------------|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0 · Bootstrap        | before 24 Jan 2026           | Project created in <b>Firebase Studio</b> from the Next.js + Genkit template (package name <code>nextn</code> , <code>.idx/</code> , <code>apphosting.yaml</code> , <code>docs/blueprint.md</code> ). <code>ProjectEscrow</code> deployed to <b>Polygon Amoy</b> on 23 Jan. Facebook login removed.         |
| 1 · MVP sprint       | 24–27 Jan 2026               | Mostly Ponmadhan: Google OAuth, Supabase schema and RLS, jobs, bids, contracts, milestones, signatures, disputes and the arbitrator "Resolution Gigs" system, wallet integration, dashboard modals, notifications, client feedback, middleware, README narrative. Work ran through the night of 26 Jan.     |
| Pause                | 27 Jan – 14 Feb              | No commits.                                                                                                                                                                                                                                                                                                 |
| 2 · Features + pivot | 14 Feb 2026                  | Parthiban: platform token wallet (buy/redeem/history) and <b>AI risk analysis</b> , homepage redesign. Merge from <code>PonmadhanD/TrustLance</code> . Ponmadhan: <b>switch to Shardeum</b> (chain 8118). Parthiban: star-rating feedback, lock-tokens modal, Next.js performance config, Gemini 2.0 Flash. |
| 3 · Hackathon finish | 15 Feb 2026, 03:00–05:17 IST | Rich profiles, mega-menu, reviews API (Parthiban). <b>Native SHM escrow</b> on Shardeum and TRT removed after its deploy failed (Ponmadhan). Final upload and cosmetic web edits. Consistent with a submission deadline for the GDG KSR Web3 Hackathon (Shardeum), which the project won.                   |
| 4 · Clean-up         | 5 Oct 2026                   | Repository moved to <code>TheParthi/TRTLance</code> ; 30,400 committed <code>node_modules</code> files and build output removed, broken submodule pointer removed, backend scripts fixed, <code>.env.example</code> and README structure added.                                                             |

### 12.2 Contributors

| Person                                      | Commits | Main contributions                                                                                                                                                                                                                         |
|---------------------------------------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Ponmadhan</b><br>(PonmadhanD)            | 17      | Core architecture; auth; Supabase schema and RLS; jobs, bids, contracts, milestones and signing; disputes and arbitrator system; LangChain dispute agent; wallet/MetaMask; ProjectEscrow; Shardeum migration and native-SHM Escrow; README |
| <b>Parthiban Gunasekaran</b><br>(TheParthi) | 18      | AI project risk analysis (Genkit/Gemini); platform token wallet; homepage, header and mega-menu; star-rating feedback; rich profiles and reviews API; performance config; repository clean-up                                              |

35 commits in total (33 regular, 2 merges) on a single `main` branch. Line counts are dominated by the Firebase Studio scaffold in the first commit.

### 12.3 Pivots

| Area               | Evolution                                                                                  | Why it matters                                                                                        |
|--------------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Blockchain network | Amoy (ProjectEscrow, 23 Jan) → Sepolia (config, 25 Jan) → <b>Shardeum 8118</b> (14–15 Feb) | The README still says Sepolia; the full escrow lives on Amoy, the app uses a minimal lock on Shardeum |

| Area          | Evolution                                                                                                                 | Why it matters                                                 |
|---------------|---------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| Token         | TRT ERC-20 → off-chain platform tokens → TRT removed after a failed Shardeum deploy ("provided fee < minimum global fee") | "TRT" in the UI and README refers to nothing on-chain          |
| Escrow design | Milestone escrow with disputes → lock-only native SHM deposit                                                             | Release and refund were lost in the pivot                      |
| Disputes      | AI + three-expert panel + admin (design doc) → single arbitrator booked through gigs                                      | Design doc and code describe different systems                 |
| AI model      | Gemini 2.5 Flash → 1.5 Flash → 2.0 Flash                                                                                  | Model id is pinned in code                                     |
| Auth          | Facebook removed; Google added; email/password kept                                                                       | README says Google only                                        |
| Repository    | <a href="#">PonmadhanD/TrustLance</a> → <a href="#">TheParthi/TRTLance</a>                                                | Origin of the broken <a href="#">TRLance</a> submodule pointer |

## 13 Implementation status

**Working** real logic against real services · **Partial** works with significant gaps · **Mock** hard-coded or simulated · **Missing** not built, or broken

### 13.1 Accounts and identity

| Capability                                                 | Status  | Notes                                          |
|------------------------------------------------------------|---------|------------------------------------------------|
| Google sign-in                                             | Working | PKCE, profile created in callback              |
| Email/password sign-in                                     | Partial | Profile insert fails with email confirmation   |
| Password reset                                             | Missing | Link points to <a href="#">#</a>               |
| Profile view ( <a href="/u/{username}">/u/{username}</a> ) | Partial | Some hard-coded stats                          |
| Profile editing                                            | Broken  | Missing import, invalid tab nesting            |
| Wallet connect                                             | Partial | No account-change handling; demo fallback      |
| Wallet linking with proof of ownership                     | Missing | Signature discarded; not permanent server-side |
| Roles (client / freelancer / arbitrator / admin)           | Partial | Per-record checks work; no admin               |

### 13.2 Marketplace

| Capability                                            | Status  | Notes                                        |
|-------------------------------------------------------|---------|----------------------------------------------|
| Post a project (wizard, signature)                    | Working | Attachments and drafts not implemented       |
| Browse / search / filter projects                     | Partial | Lists work; search and most filters inert    |
| AI risk report                                        | Partial | Real model; fabricated fallback              |
| Bid with milestones                                   | Working | Server does not validate amounts             |
| Review bids and accept                                | Working | Not atomic; project status not updated       |
| Contract signing                                      | Working | Drawn image, not cryptographic, not enforced |
| Milestone submit / revision / release                 | Working | Release repeatable; no completion state      |
| Contract chat                                         | Partial | No realtime; plaintext                       |
| Reviews and trust credits                             | Working | No party checks; farmable                    |
| Notifications (bell + page)                           | Partial | Most cross-user notifications fail RLS       |
| Bookmarks, lists, tasklists, services, quotes, groups | Mock    | Local state only                             |

### 13.3 Money

| Capability                                                  | Status    | Notes                                                       |
|-------------------------------------------------------------|-----------|-------------------------------------------------------------|
| On-chain budget lock (Shardeum)                             | Partial   | Real transfer, no release/refund, not linked to the project |
| Milestone escrow contract ( <a href="#">ProjectEscrow</a> ) | Not wired | Deployed on Amoy only                                       |
| TRT token                                                   | Missing   | Never deployed                                              |

| Capability                       | Status  | Notes               |
|----------------------------------|---------|---------------------|
| Token wallet balance and history | Working | —                   |
| Buy tokens                       | Mock    | No payment provider |
| Redeem tokens                    | Mock    | No payout           |
| PayPal, Stripe                   | Mock    | Simulated / absent  |
| Refunds                          | Missing | —                   |
| Platform fees                    | Missing | Only in UI copy     |

## 13.4 Disputes

| Capability                                   | Status  | Notes                                    |
|----------------------------------------------|---------|------------------------------------------|
| Open a dispute from the UI                   | Missing | Wizard broken; submit does not save      |
| Dispute dashboard / detail for parties       | Mock    | Two hard-coded cases                     |
| Arbitrator eligibility, slots, online status | Partial | Gate is self-editable                    |
| Arbitrator case room and chat                | Working | Only for disputes created outside the UI |
| Verdict moves funds                          | Missing | RPCs undefined                           |
| AI dispute analysis                          | Mock    | Agent exists but unused                  |
| Expert panel voting, appeals                 | Missing | Component never used                     |
| Admin console                                | Mock    | Public, hard-coded                       |

## 13.5 Platform

| Capability                       | Status  | Notes                           |
|----------------------------------|---------|---------------------------------|
| Encrypted messaging (XMTP)       | Missing | —                               |
| Production build                 | Broken  | Merge conflicts, missing module |
| Reproducible database            | Missing | Six tables not in SQL           |
| Automated tests                  | Missing | None (app or contracts)         |
| CI/CD, deployment                | Missing | Templates only                  |
| Mobile navigation, accessibility | Missing | —                               |

# 14 Security review

Ratings: **High** direct loss of funds or privilege, or unauthenticated state change · **Medium** integrity or data exposure with real impact · **Low** hardening.

The review is based on the code and the SQL in the repository. Where the live database may differ (chapter 5), the finding states both cases. No secrets were found in the git history; `.env*` files are ignored.

## 14.1 High

| ID | Finding                                                                                                                                                                                                                                                                          | Where                                                                                                                        | Fix                                                                                   |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| H1 | <b>Users can rewrite their own privileged columns</b> — balance, rating, total reviews, credits, role, membership tier, <code>tokens</code> (arbitrator gate), <code>arbitrator_status</code> , username and wallet address — directly through Supabase with the public anon key | <code>rls_policies.s</code><br><code>ql:13-19</code>                                                                         | Column-level grants or a separate server-owned table; create profiles in a trigger    |
| H2 | <b>Every user's row is public</b> , including email, phone, PayPal email, wallet and balance ( <code>SELECT USING (true)</code> ); <code>src/scripts/list-users-anon.ts</code> demonstrates it                                                                                   | <code>rls_policies.s</code><br><code>ql:9-11</code>                                                                          | Expose a <code>public_profiles</code> view with safe columns                          |
| H3 | <b>Escrowed SHM is custodial with no release path</b> ; only the deployer key can withdraw everything, and the same key is configured for mainnets                                                                                                                               | <code>Escrow.sol:45-48</code> ,<br><code>hardhat.config.js</code>                                                            | Per-milestone escrow contract with release/refund; multisig owner                     |
| H4 | <b>Free value creation</b> — token purchase needs no payment and has no upper bound; bid acceptance tops the client up to 1,000 tokens                                                                                                                                           | <code>wallet/buy/route.ts:37-71</code> ,<br><code>bids/[bidId]/accept/route.ts:57-73</code>                                  | Payment-provider webhook; remove the top-up                                           |
| H5 | <b>Release can be replayed and the amount is controlled by the parties</b> — no submission/duplicate/dispute checks; milestone amounts editable via the contracts policy; negative bids increase the client's balance                                                            | <code>release/route.ts</code> ,<br><code>bids/route.ts:20</code> ,<br><code>rls_policies.s</code><br><code>ql:187-192</code> | Milestone state machine in SQL with conditional updates; validate amounts at bid time |
| H6 | <b>Unauthenticated state-changing routes</b> — anyone can cancel any arbitrator gig (with penalty) or create an arbitrator profile for anyone                                                                                                                                    | <code>arbitrator/gigs/[gigId]/cancel</code> ,<br><code>arbitrator/apply</code>                                               | <code>getUser()</code> + ownership; server client                                     |
| H7 | <b>Double-spend races</b> in redeem, buy and accept (read-modify-write)                                                                                                                                                                                                          | <code>wallet/redeem</code> ,<br><code>wallet/buy</code> ,<br><code>bids/accept</code>                                        | Atomic SQL functions with row locks and idempotency keys                              |

## 14.2 Medium

| ID | Finding                                                                                                                                                                              | Fix                                  |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| M1 | Disputes can be opened on any project; dispute and contract chats and dispute details have no participant check in code; dispute ids are predictable ( <code>DSP-</code> + 6 digits) | Party checks; UUID ids; verified RLS |

| ID  | Finding                                                                                                                                        | Fix                                                                                |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| M2  | Dispute amount comes from the request body; resolution marks RESOLVED before funds move and ignores RPC errors; disputes do not freeze release | Derive amounts server-side; single transactional RPC; block release while disputed |
| M3  | Arbitrator assignment can be gamed — anyone can go online, and the <code>judge_bot</code> fallback username can be claimed                     | Eligibility checks; reserved usernames; conflict-of-interest rules                 |
| M4  | Reviews can be written for anyone on any project; credits and portfolio items can be farmed                                                    | Require a completed contract between the two parties; unique constraints           |
| M5  | <code>PATCH /api/jobs/[jobId]</code> writes the raw request body (mass assignment)                                                             | Field whitelist; legal status transitions only                                     |
| M6  | Bid acceptance debits money before checking for an existing contract and never rolls back                                                      | Transactional RPC                                                                  |
| M7  | AI risk reports can be forged for other users; failures produce fake reports; prompt injection                                                 | Server-only inserts; unique key; no persisted fallback                             |
| M8  | Wallet linking has no proof of ownership (signature discarded; demo address fallback)                                                          | Sign-In-With-Ethereum nonce verified on the server                                 |
| M9  | On-chain lock is not linked to the project and is never verified                                                                               | Record and verify the transaction server-side                                      |
| M10 | Known demo credentials ( <code>arbitrator@demo.com</code> / <code>password123</code> ) and login placeholders hinting at demo accounts         | Remove from production; rotate                                                     |
| M11 | <code>project_attachments</code> and <code>dispute_evidence</code> have <b>no RLS</b> — readable and writable with the anon key                | Enable RLS with party policies                                                     |

### 14.3 Low

| ID  | Finding                                                                                                                                                            |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| L1  | Raw database error messages returned to clients; the OAuth callback logs the full URL including the one-time code                                                  |
| L2  | No security headers (CSP, frame-ancestors, HSTS); auth cookies readable by JavaScript (by design of <code>@supabase/ssr</code> ), so any XSS becomes session theft |
| L3  | PostgREST filter strings built by interpolation ( <code>.or(...)</code> ) in three routes                                                                          |
| L4  | No server-side input validation library in use; unbounded strings (signatures, messages, reviews)                                                                  |
| L5  | No pagination or rate limiting (AI, buy, bid endpoints)                                                                                                            |
| L6  | Many authenticated pages are protected only in the browser; <code>/admin/*</code> not at all                                                                       |
| L7  | Contracts can be re-signed at any time                                                                                                                             |
| L8  | Google OAuth requests offline access and forces the consent screen on every login                                                                                  |
| L9  | <code>ignoreBuildErrors</code> hides type errors in production builds                                                                                              |
| L10 | Redemption requests store bank account / UPI ids in plain-text descriptions                                                                                        |

### 14.4 Priority order for remediation

- Before anything else touches real money:** H3, H4, H5, H7 — rebuild the money layer (chapter 16, phase 2).
- Before onboarding real users:** H1, H2, H6, M11 — lock down the `users` table and unauthenticated routes.

3. **Before launching disputes:** M1, M2, M3 — dispute integrity.
4. Then the remaining medium and low items.

## 15 Known issues

### 15.1 Build blockers

| # | Issue                                                                                    | Location                                                                                       | Fix                            |
|---|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|--------------------------------|
| 1 | Unresolved merge-conflict markers ( <<<<<< Updated upstream ... >>>>>> Stashed changes ) | src/app/disputes/new/page.tsx:54-171 ,<br>src/app/projects/[id]/page.tsx:50-54                 | Resolve the conflicts          |
| 2 | Import of non-existent @/components/ui/use-toast                                         | profile/edit/page.tsx:14 ,<br>profile/education-list.tsx:9 ,<br>feedback/feedback-modal.tsx:13 | Import from @/hooks/use-toast  |
| 3 | useSearchParams() without <Suspense>                                                     | disputes/new/page.tsx:96                                                                       | Wrap in Suspense               |
| 4 | TabsContent outside Tabs ; <Textarea> inside a button                                    | profile/edit/page.tsx:175-383                                                                  | Restructure the tabs           |
| 5 | Rules-of-Hooks violations                                                                | authenticated-header.tsx:133-135 ,<br>prototyper/page.tsx:37                                   | Move early returns after hooks |
| 6 | openChat and router undefined; useChat without provider                                  | projects/[id]/page.tsx                                                                         | Fix or remove the page         |
| 7 | rls_policies.sql partly UTF-16 encoded                                                   | lines 228–264                                                                                  | Re-save as UTF-8               |

### 15.2 Functional bugs

- **Accepting a bid** writes `projects.freelancer_id` (the column is `hired_freelancer_id`), so the project never becomes `in_progress` and the hired freelancer is never recorded.
- **Contracts never become completed**, so the "completed" filter is always empty and feedback timing is unenforced.
- **Two inconsistent dispute pipelines**: the contract route writes `assigned_arbitrator_id` and status `'pd'` (invalid), while resolve and the arbitrator profile read `arbitrator_id` and `UNDER_REVIEW`.
- **Gig status vocabulary** differs between routes ( `available` , `booked` , `CANCELLED` ) and the TypeScript type.
- **UI calls endpoints that do not exist**: `GET /api/bids?freelancer_id=` (405) and `GET /api/bids/{jobId}` (404).
- **Status casing**: the database stores lowercase ( `open` , `accepted` ), the UI compares uppercase ( `'OPEN'` , `'ACCEPTED'` , `'ACTIVE'` ) — open jobs show "no longer accepting", accepted bids never show escrow links.
- **Category keys** saved by post-project ( `development` , `design` ...) never match the filters on `/dashboard/jobs` ("Web Development" ...).
- **Currency labels** mix SHM, \$, ₹, TKN and INR for the same amounts; the agreement divides by 10 and calls it SHM.
- **Cross-user notifications** created with the user's session fail RLS everywhere except milestone release.
- **token\_balance** is an integer while milestone amounts are decimals.
- **Contract chat** shows the other party's messages only after a reload.
- **Risk report PDF** downloads automatically on every "Apply" click.
- **Public header** shows Log In / Sign Up to signed-in users; `/u/{username}` renders two headers and can cause a hydration mismatch.

- About **40 links** lead to routes that do not exist.

## 15.3 Documentation that contradicts the code

| Document                                  | Claim                                                                                          | Reality                                                           |
|-------------------------------------------|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| README                                    | Ethereum Sepolia testnet                                                                       | Shardeum chain 8118 (labelled mainnet)                            |
| README                                    | TRT ERC-20 token                                                                               | Never deployed; tokens are database integers                      |
| README                                    | XMTTP end-to-end encrypted messaging                                                           | Not implemented                                                   |
| README                                    | Google authentication only                                                                     | Email/password also implemented                                   |
| README                                    | One-time irreversible wallet binding                                                           | Browser-only check                                                |
| README                                    | Payments released automatically; disputes pause release                                        | Manual release; disputes don't pause it                           |
| README                                    | PayPal sandbox escrow                                                                          | Simulated                                                         |
| README                                    | Ratings and reviews out of scope; mainnet out of scope                                         | Both present                                                      |
| README                                    | Admin resolves disputes                                                                        | No admin functionality                                            |
| README                                    | Demo-ready; database schema completed                                                          | Build broken; schema incomplete                                   |
| <code>DISPUTE_RESOLUTION_SYSTEM.md</code> | AI analysis, three-expert voting, appeals, audit trail, automatic payout, encrypted evidence   | Mock screens; single arbitrator; no payout; no audit log          |
| <code>SETUP_STEPS.md</code>               | Run <code>schema.sql</code> + <code>rls_policies.sql</code> ; app on port 3000; deploy to Amoy | Migrations required; port 9002; app uses Shardeum Escrow          |
| <code>GET_PRIVATE_KEY.md</code>           | Testnet-only key                                                                               | Same key configured for mainnets                                  |
| <code>docs/blueprint.md</code>            | WalletConnect, real-time chat, AI recommendations, primary colour <code>#2563EB</code>         | None of these; primary is <code>#4F46E5</code>                    |
| <code>backend/README.md</code>            | Only ProjectEscrow; "all tables have RLS"; demo data                                           | Escrow + TrustToken exist; two tables lack RLS; demo data removed |

## 15.4 Housekeeping

- Remove dead files: `disputes/page_old.tsx`, the unused feedback modal, expert voting panel, payment demo, orphan routes.
- Remove unused dependencies (chapter 11).
- Remove Freelancer.com copy and fake statistics from the landing page, footer and metadata.
- Replace `alert()` / `confirm()` with the toast and dialog components.
- Keep one copy of each contract (`backend/contracts` is canonical).

## 16 Roadmap: from prototype to product

The prototype proves the experience. Turning it into something people can trust with real money means fixing the foundations before adding features. The phases below are ordered so that each one makes the next one safe.

### 16.1 Phase 0 — Stabilise the codebase (about 1 week)

- Resolve the merge conflicts and the missing-module imports; get `next build` green.
- Turn off `ignoreBuildErrors` / `ignoreDuringBuilds`, add ESLint, fix the type errors that surface.
- **Capture the live database** with `supabase db pull`, then rewrite `backend/supabase/` as one ordered migration history (Supabase CLI), including the six missing tables, columns, RPCs, the `avatars` bucket and realtime publication. Delete `reset.sql` and the duplicate wallet migration.
- Add a seed script that produces a usable demo (two users, a project, a contract with milestones).
- Add CI (GitHub Actions): install, typecheck, lint, build; contract compile and tests.
- Fix the README, setup guide and design docs to describe the real system (chapter 15).

### 16.2 Phase 1 — Security baseline (1–2 weeks)

- **Users table:** a `public_profiles` view for other users; privileged columns writable only by the server; profile creation in a `SECURITY DEFINER` trigger on `auth.users`.
- **RLS review of every table**, including enabling it on `project_attachments` and `dispute_evidence`, insert policies for notifications via a server function, and narrowing the contracts and proposals update policies.
- **Server-side validation** with zod on every route; whitelisted updates; consistent error responses that do not leak database messages.
- **Authenticate every route**; replace the browser client in server routes; add rate limiting to AI, payment and bidding endpoints.
- **Sign-In-With-Ethereum** for wallet linking: server nonce, signature verification, immutable binding.
- Security headers (CSP, frame-ancestors, HSTS); remove demo credentials.

### 16.3 Phase 2 — One trustworthy money layer (3–4 weeks)

Today there are five unreconciled value stores. Choose **one** model and remove the rest.

#### Option A — On-chain escrow (recommended)

Deploy an upgraded `ProjectEscrow` on Shardeum: the client funds all milestones at hiring, approval releases each milestone to the freelancer's wallet, disputes freeze funds and the arbiter settles with a percentage split. The database mirrors chain state by indexing contract events (never by trusting the browser). Keeps TrustLance's core promise and its Shardeum relationship. Needs: contract tests, an external audit, a multisig arbiter, gas/UX handling.

#### Option B — Off-chain ledger with real payments

Keep platform tokens but make them real: payments through a provider such as Razorpay with verified webhooks, payouts through its payout API, and every balance change in atomic SQL functions with an immutable ledger. Simpler for non-crypto users, but the platform holds the money, which weakens the "trust-first" story and brings payment-regulation obligations.

Either way:

- One milestone model: `pending → funded → submitted → approved → paid` (plus `disputed`, `refunded`), enforced in SQL or on-chain, idempotent.
- Remove the auto top-up, the mock purchase and every simulated payment path.
- Refunds, cancellations and deadlines (auto-release after a review window).
- A clear fee model, if any, implemented in one place.
- Reconciliation reports that compare ledger, contracts and chain.

## 16.4 Phase 3 — Disputes that work end to end (2–3 weeks)

- Parties open a dispute from the contract (fixed wizard), attach evidence to Supabase Storage with RLS, and follow it on a real dispute page.
- Arbitrator eligibility based on a server-owned stake or reputation, conflict-of-interest checks, domain matching, response deadlines and escalation.
- The AI agent becomes a Genkit flow that reads the contract, submissions, chat and evidence and produces a **recommendation** shown to the arbitrator.
- Verdicts settle funds through the Phase 2 money layer in one transaction; the contract leaves `disputed`.
- An append-only audit trail for every dispute event; an admin console behind a real admin role.

## 16.5 Phase 4 — Communication (1–2 weeks)

- Decide between XMTP (wallet-to-wallet, end-to-end encrypted — the original promise) and Supabase Realtime chat with clear privacy wording.
- Real-time contract chat, unify the inbox, delete the mock widget.
- Fix notifications: server-side creation, valid types and links, email for important events.

## 16.6 Phase 5 — Product polish (2–3 weeks)

- Mobile navigation, keyboard-accessible menus, labelled controls, focus states, contrast; replace `alert()` / `confirm()`.
- Remove or finish secondary pages (bookmarks, lists, services, quotes ...); remove all fake statistics and Freelancer.com copy; consistent currency labels and status casing.
- Search and filters on real data with pagination; project attachments; drafts.
- Onboarding: choose client/freelancer intent, complete profile, connect wallet.
- Make the AI risk report trustworthy (chapter 8).

## 16.7 Phase 6 — Production readiness

- Hosting (Firebase App Hosting or Vercel) with environment secrets, preview deployments and monitoring (error tracking, logs, uptime).
- Automated tests: API integration tests against a local Supabase, contract tests with coverage, end-to-end tests for the core journey.
- Contract audit and mainnet deployment plan with key management (hardware wallet / multisig).
- Terms of service, privacy policy, data retention, and KYC/AML review if real money moves.

## 16.8 Suggested first sprint

1. Fix the build (Phase 0, items 1–2).
2. Pull and commit the real schema.
3. Lock down `users` and the two unauthenticated routes (H1, H2, H6).
4. Make milestone release idempotent and remove the auto top-up (H4, H5).
5. Correct the README.

After this sprint the codebase is reproducible, safe to demo with real users, and ready for the money-layer decision.

# 17 Appendix

## 17.1 Glossary

| Term                         | Meaning in TrustLance                                                                                                                     |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Project / job                | A row in <code>projects</code> ; "job" in API paths and UI, "project" in the database                                                     |
| Bid / proposal               | A row in <code>proposals</code> ; "bid" in the API                                                                                        |
| Contract                     | Created when a bid is accepted; holds milestones, signatures and locked amount                                                            |
| Milestone                    | An entry in <code>contracts.milestones</code> (JSON array), addressed by index; its work is tracked in <code>milestone_submissions</code> |
| Token / TKN                  | Platform credit stored in <code>user_wallets.token_balance</code> ; 1 token = ₹10                                                         |
| TRT                          | TrustToken, an ERC-20 that was planned but never deployed                                                                                 |
| SHM                          | Shardeum's native coin, used for the on-chain budget lock                                                                                 |
| Escrow (on-chain)            | <code>Escrow.sol</code> on Shardeum chain 8118 ( <code>lockFunds</code> )                                                                 |
| ProjectEscrow                | Full milestone escrow contract, deployed on Polygon Amoy, not used by the app                                                             |
| Arbitrator / Resolution Gigs | Users with $\geq 3,000$ tokens who book slots and decide disputes                                                                         |
| Trust credits                | <code>users.total_credits</code> , +30 per client review                                                                                  |
| RLS                          | Supabase row-level security policies                                                                                                      |
| Service role                 | Supabase secret key that bypasses RLS (server only)                                                                                       |

## 17.2 Key files

| Area                                   | File                                                                                                                 |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| App shell and providers                | <code>src/app/layout.tsx</code> , <code>src/components/layout/conditional-layout.tsx</code>                          |
| Route protection                       | <code>src/middleware.ts</code>                                                                                       |
| Auth and profile creation              | <code>src/contexts/auth-context.tsx</code> , <code>src/app/auth/callback/route.ts</code>                             |
| Supabase clients                       | <code>src/lib/supabase.ts</code> , <code>src/lib/supabase/{client,server,admin}.ts</code>                            |
| Wallet                                 | <code>src/contexts/wallet-context.tsx</code> , <code>src/components/wallet/*</code> , <code>src/lib/config.ts</code> |
| Project posting and on-chain lock      | <code>src/app/post-project/page.tsx</code>                                                                           |
| Job detail, AI report, bidding, accept | <code>src/app/job/[jobId]/page.tsx</code> , <code>src/components/RiskAnalysisModal.tsx</code>                        |
| Contract workspace                     | <code>src/app/contracts/[contractId]/page.tsx</code>                                                                 |
| Bid acceptance (funding)               | <code>src/app/api/bids/[bidId]/accept/route.ts</code>                                                                |
| Milestone release (payment)            | <code>src/app/api/contracts/[contractId]/milestones/[index]/release/route.ts</code>                                  |
| Token wallet                           | <code>src/app/api/wallet/*</code> , <code>src/app/profile/{wallet,token-store,token-redeem}</code>                   |

| Area            | File                                                                                                                                                                                   |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Arbitration     | <code>src/app/resolution-gigs/**</code> , <code>src/app/api/arbitrator/**</code> ,<br><code>src/app/api/disputes/**</code> , <code>src/lib/arbitrator.ts</code>                        |
| AI              | <code>src/ai/genkit.ts</code> , <code>src/ai/flows/*</code> , <code>src/app/api/ai/project-risk/route.ts</code> ,<br><code>backend/ai/dispute-agent.js</code>                          |
| Smart contracts | <code>backend/contracts/{Escrow,ProjectEscrow,TrustToken}.sol</code> ,<br><code>backend/hardhat.config.js</code> , <code>backend/scripts/*</code> , <code>backend/deployments/*</code> |
| Database        | <code>backend/supabase/schema.sql</code> , <code>rls_policies.sql</code> , <code>feedback_schema.sql</code> ,<br><code>migrations/*</code>                                             |

## 17.3 Frontend → API map

| API                                                                                                        | Called from                                                                                                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>GET /api/jobs</code>                                                                                 | dashboard, dashboard/jobs, find-jobs, my-projects                                                                                                                                                                                     |
| <code>POST /api/jobs</code>                                                                                | post-project                                                                                                                                                                                                                          |
| <code>GET /api/jobs/[id]</code> , <code>GET /api/jobs/[id]/bids</code>                                     | job detail                                                                                                                                                                                                                            |
| <code>POST /api/bids</code> , <code>GET /api/bids/my-bids</code> , <code>POST /api/bids/[id]/accept</code> | job detail, my-projects                                                                                                                                                                                                               |
| <code>GET /api/contracts</code>                                                                            | contracts list                                                                                                                                                                                                                        |
| Contract detail, sign, messages, submit, feedback, release                                                 | contract workspace                                                                                                                                                                                                                    |
| <code>POST /api/reviews</code>                                                                             | contract workspace (feedback modal)                                                                                                                                                                                                   |
| <code>POST /api/ai/project-risk</code>                                                                     | job detail                                                                                                                                                                                                                            |
| <code>PUT /api/profile/update</code>                                                                       | profile edit                                                                                                                                                                                                                          |
| <code>/api/wallet/*</code>                                                                                 | profile wallet, token store, token redeem                                                                                                                                                                                             |
| <code>/api/arbitrator/profile</code> ,<br><code>.../status</code> , <code>.../gigs/book</code>             | resolution-gigs                                                                                                                                                                                                                       |
| <code>GET /api/disputes/[id]</code> ,<br>messages, resolve                                                 | arbitrator room                                                                                                                                                                                                                       |
| Never called                                                                                               | <code>GET/POST /api/disputes</code> , <code>/api/arbitrator/apply</code> ,<br><code>/api/arbitrator/gigs/[id]/cancel</code> , <code>PATCH /api/jobs/[id]</code> , <code>POST /api/projects</code> , <code>/api/feedback/submit</code> |

## 17.4 Contract addresses

| Contract      | Network              | Address                                                 | Deployer / owner                                        |
|---------------|----------------------|---------------------------------------------------------|---------------------------------------------------------|
| ProjectEscrow | Polygon Amoy (80002) | <code>0x2DC618147ee8360CD83d5ed3368525B920F2712F</code> | <code>0xD62Cd999B20A643469a40C1a9B27be1E9F6D2A79</code> |
| Escrow        | Shardeum (8118)      | <code>0x2DC618147ee8360CD83d5ed3368525B920F2712F</code> | same                                                    |

On-chain state was not verified for this document; the addresses come from `backend/deployments/*.json`.

## 17.5 About this document

- Written for the TrustLance team as the baseline for further development, from a full read of the repository at commit `5d9c2268` (5 October 2026).
- Source: `docs/technical-reference/*.md` (one file per chapter), `style.css`, `meta.json`. Rebuild the PDF with `node build.mjs` (instructions at the top of that file).
- When the code changes, update the matching chapter and the status matrix, bump the version in `meta.json` and rebuild.